

este modelo los procedimientos PONE_EN_COLA y QUITA_DE_COLA se pueden escribir de tal manera que su ejecución se realice en un número constante de pasos.

Hay una sutileza que surge en la representación de la figura 2.21 y en cualquier variante menor de esta estrategia (es decir, si *C.post* apunta a una posición adelantada con respecto al último elemento, en el sentido de las manecillas del reloj, y no a ese mismo elemento). El problema reside en que no hay manera de distinguir entre una cola vacía y una que ocupa el círculo completo, a menos que se mantenga un bit que sea verdadero si, y sólo si, la cola está vacía. Si no se está dispuesto a mantener ese bit, se debe evitar que la cola llegue a llenar todo el arreglo.

Para comprender la razón de esto, supóngase que la cola de la figura 2.21 tiene *long_máx* elementos. Entonces *C.post* debería apuntar una posición adelante de *C.ant* en el sentido contrario al de las manecillas del reloj. ¿Y qué ocurriría si la cola estuviera vacía? Para ver cómo se representa una cola vacía, considérese primero una que tenga un solo elemento. Entonces *C.ant* y *C.post* apuntan a la misma posición. Si en estas condiciones se suprime ese elemento, *C.ant* avanza una posición en el sentido de las manecillas del reloj, formando una cola vacía. Así, en una cola vacía, *C.post* está a una posición de *C.ant* en sentido contrario al de las manecillas del reloj; es decir, está exactamente en la misma posición relativa que ocuparía si la cola tuviera *long_máx* elementos. Resulta evidente, pues, que aun cuando el arreglo tenga *long_máx* lugares, no se puede permitir que la cola crezca más que *long_máx*-1, a menos que se introduzca un mecanismo para distinguir las colas vacías.

```
procedure ANULA ( var C: COLA );
```

```
begin
```

```
  new(C.ant); { crea la celda de encabezamiento }
```

```
  C.ant↑.sig := nil;
```

```
  C.post := C.ant { el encabezamiento es la primera y la última celdas }
```

```
end; { ANULA }
```

```
function VACIA ( C: COLA ): boolean;
```

```
begin
```

```
  if C.ant = C.post then
```

```
    return (true)
```

```
  else
```

```
    return (false)
```

```
end; { VACIA }
```

```
function FRENTE ( C: COLA ): tipo_elemento;
```

```
begin
```

```
  if VACIA(C) then
```

```
    error('la cola está vacía')
```

```
  else
```

```
    return ( C.ant↑.sig↑.elemento )
```

```
end; { FRENTE }
```

```

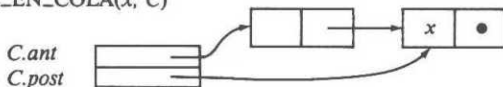
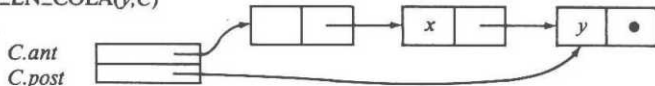
procedure PONE_EN_COLA ( x: tipo_elemento; var C: COLA );
begin
    new ( C.post↑.sig ); { agrega una nueva celda en el extremo
                          posterior de la cola }
    C.post := C.post↑.sig;
    C.post↑.elemento := x;
    C.post↑.sig := nil
end; { PONE_EN_COLA }

procedure QUITA_DE_COLA ( var C: COLA );
begin
    if VACIA(C) then
        error ('la cola está vacía')
    else
        C.ant := C.ant↑.sig
    end; { QUITA_DE_COLA }

```

Fig. 2.19. Implantación de mandatos para colas.

ANULA (C)

PONE_EN_COLA(*x*, C)PONE_EN_COLA(*y*, C)

QUITA_DE_COLA(C)

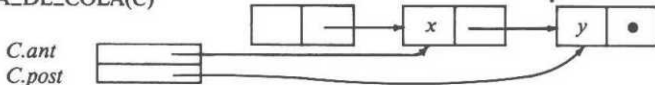


Fig. 2.20. Sucesión de operaciones con colas.

Se escribirán ahora los cinco mandatos para colas usando esta representación. Formalmente, las colas se definen:

```

type
  COLA = record
    elementos: array[1..long_máx] of tipo_elemento;
    ant, post: integer
  end;

```

Los mandatos aparecen en la figura 2.22. La función *suma_uno(i)* suma uno a la posición *i*, en el sentido circular.

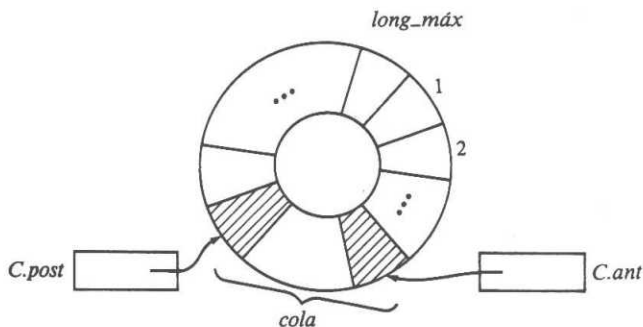


Fig. 2.21. Realización circular para colas.

2.5 Correspondencias

Una *correspondencia* o *memoria asociativa* es una función de elementos de un tipo llamado *tipo_dominio*, a elementos de otro tipo (quizás el mismo) llamado *tipo_contradominio*. Se expresa el hecho de que la correspondencia *M* asocia el elemento *r* de tipo *tipo_contradominio* con el elemento *d* de tipo *tipo_dominio* por $M(d) = r$.

Ciertas correspondencias, como $\text{cuadrado}(i) = i^2$, se pueden obtener con facilidad como una función de Pascal por medio de una expresión aritmética u otro medio simple de calcular $M(d)$ a partir de *d*. Sin embargo, en el caso de muchas correspondencias, la única manera clara de describir $M(d)$ consiste en almacenar para cada *d* el valor de $M(d)$. Por ejemplo, para aplicar una función de nómina que asocie a cada empleado un salario semanal, parece obligatorio almacenar el salario actual de cada empleado. En lo que resta de esta sección se describirá un método para implantar funciones como la función «nómina».

Considérese qué operaciones es deseable realizar en una correspondencia *M*. Dado un elemento *d* de un tipo dominio, se podría desear obtener $M(d)$ o saber si $M(d)$ está definida (es decir, si *d* pertenece actualmente al dominio de *M*). O quizá

```

function suma_uno ( i : integer ) : integer;
  begin
    return ((i mod long_máx) + 1)
  end; { suma_uno }

procedure ANULA ( var C: COLA );
  begin
    C. ant := 1;
    C. post := long_máx
  end; { ANULA }

function VACIA ( var C: COLA ) : boolean;
  begin
    if suma_uno(C. post) = C.ant then
      return (true)
    else
      return (false)
    end; { VACIA }

function FRENTE ( var C: COLA ) : tipo_elemento;
  begin
    if VACIA (C) then
      error ('la cola está vacía')
    else
      return (C.elementos[C.ant])
    end; { FRENTE }

procedure PONE_EN_COLA ( x: tipo_elemento; var C: COLA );
  begin
    if suma_uno(suma_uno(C.post)) = C.ant then
      error ('la cola está llena')
    else begin
      C.post := suma_uno(C.post);
      C.elementos[C.post] := x
    end
  end; { PONE_EN_COLA }

procedure QUITA_DE_COLA ( var C: COLA );
  begin
    if VACIA (C) then
      error ('la cola está vacía')
    else
      C.ant := suma_uno(C.ant)
    end; { QUITA_DE_COLA }

```

Fig. 2.22. Realización circular de colas.

se desee introducir nuevos elementos en el dominio actual de M y establecer sus valores asociados de contradominio. Como solución distinta, se podría desear cambiar el valor de $M(d)$. También se necesita una manera de asignar valor inicial a una correspondencia como la *correspondencia nula* o vacía, cuyo dominio está vacío. Estas operaciones se resumen en los tres mandatos siguientes:

1. **ANULA** (M). Hace que M sea la correspondencia nula.
2. **ASIGNA** (M, d, r). Define $M(d)$ como r , tanto si $M(d)$ está definido previamente como si no.
3. **CALCULA** (M, d, r). Devuelve verdadero y da a r el valor $M(d)$ si este último está definido; en caso contrario, devuelve falso.

Realización de correspondencias mediante arreglos

Muchas veces, el tipo dominio de una correspondencia será un tipo elemental que pueda usarse como tipo índice de un arreglo. En Pascal los tipos índice incluyen todos los subcontradominios finitos de los enteros, como 1..100 ó 17..23, el tipo *char* y los subcontradominios de *char*, como 'A'..'Z', y los tipos enumerados como (norte, sur, este, oeste). Por ejemplo, un programa para descifrar textos podría guardar una correspondencia *cif*, con 'A'..'Z' como sus tipo_dominio y tipo_contradominio, de modo que *cif(letra_texto)* sea la letra que en un momento dado se suponga que representa a la letra *letra_texto*.

Tales correspondencias se pueden realizar sencillamente por medio de arreglos, siempre que haya un valor del tipo_contradominio que signifique «indefinido». Por ejemplo, la correspondencia *cif* anterior podría definirse con *char* como su tipo_contradominio, en lugar de 'A'..'Z', y '?' podría usarse para denotar «indefinido».

Supóngase que los tipos dominio y contradominio son tipo_dominio y tipo_contradominio, respectivamente, y que tipo_dominio es un tipo de Pascal básico. Entonces, se puede definir el tipo CORRESPONDENCIA (estrictamente hablando, correspondencia de tipo_dominio a tipo_contradominio) por la declaración:

```
type
  CORRESPONDENCIA = array[tipo_dominio] of tipo_contradominio;
```

En el supuesto de que «indefinido» sea una constante de tipo_contradominio y que primer_valor y último_valor sean el primero y el último valores de tipo_dominio †, es posible implantar los tres mandatos en correspondencias como en la figura 2.23.

Realización de correspondencias mediante listas

Existen muchas aplicaciones posibles de las correspondencias con dominios finitos. Por ejemplo, las tablas de dispersión (*hash*) son una excelente alternativa en mu-

† Por ejemplo, primer_valor = 'A' y último_valor = 'Z', si el tipo_dominio es 'A'..'Z'.

chas situaciones, pero su análisis se deja para el capítulo 4. Cualquier correspondencia con un dominio finito se puede representar mediante la lista de pares (d_1, r_1) , (d_2, r_2) , ..., (d_k, r_k) , donde $d_1, d_2, \dots, d_k$ son todos los miembros actuales del dominio, y r_i es el valor que la correspondencia asocia a d_i , para $i = 1, 2, \dots, k$. Se puede entonces usar cualquier implantación de listas que se elija para representar esa lista de pares.

Para precisar más, el tipo de datos abstracto CORRESPONDENCIA se puede aplicar por medio de listas de tipo_elemento si se define

```

type
  tipo_elemento = record
    dominio: tipo_dominio;
    contradominio: tipo_contradominio
  end;

```

y luego se define CORRESPONDENCIA como se haría con el tipo LISTA (de tipo_elemento) en la realización de listas elegida. En la figura 2.24 se definen los tres mandatos de correspondencias en función de mandatos sobre el tipo LISTA.

```

procedure ANULA ( var M: CORRESPONDENCIA );
var
  i: tipo_dominio;
begin
  for i := primer_valor to último_valor do
    M[i] := indefinido
  end; { ANULA }

procedure ASIGNA ( var M: CORRESPONDENCIA;
  d: tipo_dominio; r: tipo_contradominio);
begin
  M[d] := r
end; { ASIGNA }

function CALCULA ( var M: CORRESPONDENCIA;
  d: tipo_dominio; var r: tipo_contradominio): boolean;
begin
  if M[d] = indefinido then
    return (false)
  else begin
    r := M[d];
    return (true)
  end
end; { CALCULA }

```

Fig. 2.23. Realización de correspondencias mediante arreglos.

2.6 Pilas y procedimientos recursivos

Una aplicación importante de las pilas se da en la aplicación de procedimientos recursivos en los lenguajes de programación. La *organización a tiempo de ejecución* de uno de tales lenguajes es el conjunto de estructuras de datos usadas para representar los valores de las variables de un programa durante su ejecución. Todo lenguaje que, como Pascal, permita procedimientos recursivos, utiliza una pila de *registros de activación* para registrar los valores de todas las variables que pertenecen a cada procedimiento activo de un programa. Cuando se llama a un procedimiento P , se coloca en la pila un nuevo registro de activación para P , con independencia de si ya existe en ella otro registro de activación para ese mismo procedimiento. Cuando P vuelve, su registro de activación debe estar en el tope de la pila, puesto que P no puede volver si no lo han hecho previamente todos los procedimientos a los que P ha llamado. Así, se puede sacar de la pila el registro de activación correspondiente a la llamada actual de P y hacer que el control regrese al punto en el que P fue llamado (este punto, conocido como *dirección de retorno*, se colocó en el registro de activación de P al llamar a este procedimiento).

```
procedure ANULA ( var M: CORRESPONDENCIA );
{ igual que para listas }
```

```
procedure ASIGNA ( var M: CORRESPONDENCIA;
  d: tipo_dominio; r: tipo_contradominio);
var
  x: tipo_elemento; { el par (d, r) }
  p: posición; { usada para ir de la primera a la última posición de
    la lista M }
begin
  x.dominio := d;
  x.contradominio := r;
  p := PRIMERO(M);
  while p <> FIN(M) do
    if RECUPERA(p, M).dominio = d then
      SUPRIME(p, M) { elimina el elemento con dominio d }
    else
      p := SIGUIENTE(p, M);
  INSERTA (x, PRIMERO(M), M) { coloca (d, r) al inicio de la lista }
end; { ASIGNA }
```

```
function CALCULA ( var M: CORRESPONDENCIA;
  d: tipo_dominio; var r: tipo_contradominio): boolean;
var
  p: posición;
begin
  p := PRIMERO (M);
  while p <> FIN(M) do begin
    if RECUPERA(p, M).dominio = d then begin
```

```

    r := RECUPERA(p, M). contradominio;
    return (true)
end;
p := SIGUIENTE(p, M)
end;
return (false) { si d no está en el dominio }
end; { CALCULA }

```

Fig. 2.24. Realización de correspondencias en función de listas.

La recursión simplifica la estructura de muchos programas. Sin embargo, en algunos lenguajes las llamadas a procedimientos tienen un costo mucho mayor que el de las proposiciones de asignación, de modo que la ejecución de un programa puede resultar más rápida, en un factor constante considerable, si se eliminan las llamadas recursivas. Al decir esto no se propugna la eliminación habitual de la recursión o de otras llamadas a procedimientos; es frecuente que la sencillez estructural justifique el tiempo de ejecución. Sin embargo, podría ser deseable eliminar la recursión en las porciones de los programas que se ejecutan con mayor frecuencia, y el propósito del análisis que sigue es ilustrar cómo se pueden convertir los procedimientos recursivos en procedimientos no recursivos mediante la introducción de una pila definida por el programador.

Ejemplo 2.3. Considérense dos soluciones, una recursiva y otra no recursiva, a una versión simplificada del clásico *problema de la mochila*, en el cual se da un objetivo o y una colección de pesos $p_1, p_2, \dots, p_n$ (enteros positivos). Se pide determinar si existe una selección de pesos que totalice exactamente o . Por ejemplo, si $o = 10$ y los pesos son 7, 5, 4, 4 y 1, se pueden elegir el segundo, el tercero y el quinto, ya que $5 + 4 + 1 = 10$.

La justificación del nombre «problema de la mochila» es que se desea llevar en la espalda no más de o kilogramos, y se tiene para elegir un conjunto de objetos de pesos dados. Se supone, además, que la utilidad de los objetos es proporcional a su peso $\dagger$, y por ello se desea cargar la mochila con un peso lo más cercano posible al objetivo.

En la figura 2.25 se puede ver una función *mochila* que opera en una matriz

pesos: array[1..n] of integer.

La llamada a *mochila*(s, i) determina si existe una colección de los elementos entre *peso*[i] y *peso*[n] que sume exactamente s , e imprime sus pesos de ser tal el caso. Lo primero que *mochila* hace es determinar si puede responder de inmediato. Específicamente, si $s = 0$, entonces el conjunto vacío de pesos es una solución. Si $s < 0$, no puede haber solución, y si $s > 0$ e $i > n$, entonces ya no hay más pesos que considerar y, por tanto, no se puede encontrar una suma de pesos igual a s .

Si no se tiene ninguno de estos casos, entonces sencillamente se hace la llamada a *mochila*($s - p_i, i + 1$), para ver si existe una solución que incluya a p_i . Si esa solu-

$\dagger$ En el «verdadero» problema de la mochila, se dan valores de utilidad y pesos, y se pide maximizar la utilidad de los objetos transportados, con una restricción del peso.

ción existe, todo el problema está resuelto y la solución incluye a p_i , de modo que éste se imprime. Si no hay solución, se efectúa la llamada a *mochila*($s, i + 1$), para ver si existe una solución que no use p_i . □

Eliminación de la recursión de cola

A menudo, es posible eliminar en forma mecánica la última llamada que un procedimiento se hace a sí mismo. Si un procedimiento $P(x)$ tiene como último paso una llamada a $P(y)$, entonces es posible reemplazar la llamada a $P(y)$ por una asignación $x := y$, seguida de un salto al principio del código de P . Aquí, y puede ser una expresión, pero x debe ser un parámetro pasado por valor, de modo que su valor se almacena en una localidad de memoria privada de esta llamada a P †. Por supuesto, P podría tener más de un parámetro, en cuyo caso se tratarían exactamente igual que x e y .

```

function mochila ( objetivo: integer; candidato: integer ): boolean;
begin
(1)   if objetivo = 0 then
(2)       return ( true )
(3)   else if (objetivo < 0) or (candidato > n) then
(4)       return(false)
      else { considera soluciones con y sin candidato }
(5)   if mochila(objetivo-pesos[candidato], candidato + 1) then
      begin
(6)       writeln(pesos [candidato] );
(7)       return (true)
      end
      else { la única solución posible es sin candidato }
(8)   return (mochila(objetivo, candidato + 1))
end; { mochila }

```

Fig. 2.25. Solución recursiva al problema de la mochila.

Este cambio funciona, porque volver a ejecutar P con el nuevo valor de x equivale exactamente a llamar a $P(y)$ y luego volver de esa llamada. Obsérvese que el hecho de que algunas de las variables locales de P tengan valores en la segunda llamada no tiene consecuencias. P no podría usar ninguno de esos valores, pues si se hubiera llamado a $P(y)$ como se intentaba en un principio, esos valores no habrían estado definidos.

En la figura 2.25 se ilustra otra variante de la recursión de cola; ahí, el último paso de la función *mochila* sólo devuelve el resultado de llamarse a sí misma con otros parámetros. En una situación tal, suponiendo que los parámetros se pasan por

† Alternativamente, x podría pasarse por referencia si y es x .

valor (o por referencia, si el mismo parámetro es el que se pasa a la llamada), se puede reemplazar la llamada por asignaciones a los parámetros y un salto al principio de la función. En el caso de la figura 2.25, se puede reemplazar la línea (8) por

```
candidato := candidato + 1;
goto principio
```

donde *principio* es una etiqueta que se va a asignar a la proposición (1). Obsérvese que no se necesita ningún cambio a *objetivo*, puesto que su valor pasa intacto como primer parámetro. De hecho, se puede advertir que, al no haber cambiado *objetivo*, las pruebas de las proposiciones (1) y (3) que lo incluyen están destinadas a fallar y, por tanto, es posible omitir las líneas (1) y (2), realizar sólo la prueba *candidato* > *n* en la línea (3) y luego proseguir directamente a la línea (5).

Eliminación completa de la recursión

El procedimiento de eliminación de la recursión de cola suprime la recursión por completo sólo cuando la llamada recursiva se encuentra al final del procedimiento y tiene la forma adecuada. Existe un enfoque más general que convierte cualquier procedimiento (o función) recursivo en no recursivo, pero que introduce una pila definida por el programador. En general, una celda de esa pila contendrá:

1. los valores actuales de los parámetros del procedimiento;
2. los valores actuales de todas las variables locales del procedimiento, y
3. una indicación de la dirección de retorno, esto es, del lugar a donde deberá volver el control cuando la invocación actual del procedimiento termine.

En el caso de la función *mochila*, se puede hacer algo más sencillo. Primero, se observa que cada vez que se hace una llamada (que implica meter un registro a la pila), *candidato* se incrementa en 1. Así pues, se puede dejar *candidato* como una variable global, incrementando su valor en 1 cada vez que se mete un registro en la pila y disminuyéndolo en 1 cada vez que se saca un registro.

Una segunda simplificación posible consiste en mantener dentro de la pila una «dirección de retorno» modificada. En términos estrictos, la dirección de retorno para esta función es un lugar de otro procedimiento que llama a *mochila*, la llamada de la línea (5) o la llamada de la línea (8). Estas tres posibilidades se representan por una variable «de estado» que tiene uno de los tres valores siguientes:

1. *ninguno*, que indica que la llamada procede de fuera de la función *mochila*.
2. *incluido*, que indica la llamada de la línea (5), la cual incluye *pesos[candidato]* dentro de la solución, o
3. *excluido*, que indica la llamada de la línea (8), la cual excluye a *pesos[candidato]*.

Si se almacena esta variable de estado como la dirección de retorno, se puede manejar *objetivo* como una variable global. Cuando el estado cambia de *ninguno* a *incluido*, se sustrae *pesos[candidato]* de *objetivo*, y se suma de nuevo cuando el estado

cambia de *incluido* a *excluido*. Como ayuda para representar el efecto del retorno de *mochila* cuando indican si se ha hallado la solución, se usa una variable global *bandera_éxito*. Una vez que *bandera_éxito* toma el valor verdadero, lo conserva y hace que se saquen de la pila los registros, imprimiendo aquellos pesos que tienen asociado un estado *incluido*. Con estas modificaciones, se puede declarar la pila como una pila de estados, mediante.

type

estados = (*ninguno*, *incluido*, *excluido*);

PILA = { declaración adecuada para una pila de estados }

La figura 2.26 muestra el procedimiento resultante no recursivo *mochila*, que opera en un arreglo de *pesos* como antes. Aunque este procedimiento puede ser más rápido que la función *mochila* original, es claramente más largo y más difícil de entender. Por ello sólo se debe eliminar la recursión cuando la velocidad sea muy importante.

procedure *mochila* (*objetivo*: integer);

var

candidato: integer;

bandera_éxito: boolean;

P: PILA;

begin

candidato := 1;

bandera_éxito := false;

ANULA(*P*);

METE(*ninguno*, *P*); { asigna valor inicial a la pila considerando *pesos[1]* }

repeat

if *bandera_éxito* **then begin**

{ saca de la pila e imprime los pesos incluidos en la solución }

if TOPE(*P*) = *incluido* **then**

writeln (*pesos[candidato]*);

candidato := *candidato* - 1;

SACA(*P*)

end

else if *objetivo* = 0 **then begin** { se encontró la solución }

bandera_éxito := true;

candidato := *candidato* - 1;

SACA(*P*)

end

else if (((*objetivo* < 0) and (TOPE(*P*) = *ninguno*))

or (*candidato* > *n*)) **then begin**

{ no hay solución posible con las elecciones hechas }

candidato := *candidato* - 1;

SACA(*P*)

```

end
else { aún no hay decisión, se considera el estado del candidato actual }
if TOPE(P) = ninguno then begin { se intenta primero incluyendo al
    candidato }
    objetivo := objetivo - pesos[candidato];
    candidato := candidato + 1;
    SACA(P); METE(incluido, P); METE (ninguno, P)
end
else if TOPE(P) = incluido then begin { se intenta excluyendo al
    candidato }
    objetivo := objetivo + pesos[candidato];
    candidato := candidato + 1;
    SACA(P); METE(excluido, P); METE(ninguno, P)
end
else begin { TOPE(P) = excluido; la elección actual no da resultado }
    SACA(P);
    candidato := candidato - 1
end
until VACIA (P)
end; { mochila }

```

Fig. 2.26. Procedimiento no recursivo para el problema de la mochila.

Ejercicios

- 2.1 Escribase un programa que imprima los elementos de una lista. En los ejercicios siguientes usense operaciones con listas para realizar los programas.
- 2.2 Escribanse programas para insertar, suprimir y localizar un elemento en una lista clasificada, usando realizaciones de
 - a) arreglos
 - b) apuntadores y
 - c) cursores.
 ¿Cuál es el tiempo de ejecución de cada uno de estos programas?
- 2.3 Escribase un programa para intercalar
 - a) dos listas clasificadas,
 - b) n listas clasificadas.
- 2.4 Escribase un programa para concatenar una lista de listas.
- 2.5 Supóngase que se desea manipular polinomios de la forma $p(x) = c_1x^{e_1} + c_2x^{e_2} + \dots + c_nx^{e_n}$, donde $e_1 > e_2 > \dots > e_n \geq 0$. Un polinomio de ese tipo se puede representar mediante una lista enlazada en la que cada celda

tiene tres campos: uno para el coeficiente c_i , otro para el exponente e_i , y otro para el apuntador a la siguiente celda. Escribese un programa para diferenciar polinomios representados de esta manera.

- 2.6 Escribáse programas para sumar y multiplicar polinomios similares a los del ejercicio 2.5. ¿Cuál es el tiempo de ejecución de los programas en función del número de términos?

- *2.7 Supóngase que se declaran celdas mediante

```

type
  tipo_celda = record
    bit: 0..1;
    sig: ↑ tipo_celda
  end;

```

Un número binario $b_1b_2 \dots b_n$, donde cada b_i es 0 ó 1, tiene el valor numérico $\sum_{i=1}^n b_i 2^{n-i}$. Este número se puede representar por la lista $b_1, b_2, \dots, b_n$.

Esta lista, a su vez, puede representarse como una lista enlazada de celdas de tipo `tipo_celda`. Escribáse un procedimiento *incrementa(número_bin)* que sume uno al número binario apuntado por *número_bin*. Sugerencia: Hágase *incrementa* recursivo.

- 2.8 Escribáse un procedimiento para intercambiar los elementos de las posiciones p y SIGUIENTE(p) de una lista enlazada sencilla.

- *2.9 El siguiente procedimiento se hizo con el propósito de suprimir todas las apariciones de un elemento x de una lista L . Explíquese por qué no siempre funciona y sugiérase una manera de arreglarlo de modo que realice la tarea para la que fue propuesto.

```

procedure supprime (x:tipo_elemento; var L: LISTA);
var
  p: posición;
begin
  p:= PRIMERO (L);
  while p <> FIN (L) do begin
    if RECUPERA (p, L) = x then
      SUPRIME (p, L);
      p:= SIGUIENTE (p, L)
    end
  end; { supprime }

```

- 2.10 Se desea almacenar una lista en un arreglo A cuyas celdas contienen dos campos: *datos*, que contiene un elemento, y *posición*, que da la posición (entera) del elemento. Un entero *último* indica que la lista ocupa las posiciones $A[1]$ a $A[\text{último}]$. El tipo LISTA se puede definir como:

```

type
  LISTA = record
    último: integer;
    elementos: array[1..long_máx] of record
      datos: tipo_elemento;
      posición: integer
    end
  end;

```

Escribase un procedimiento SUPRIME(p, L) que elimine el elemento de la posición p . Inclúyanse todas las verificaciones necesarias para detectar errores.

- 2.11 Supóngase que L es una LISTA y que p, q y r son posiciones. En función de la longitud n de la lista L , determínese cuántas veces se ejecutan en el siguiente programa las funciones PRIMERO, FIN y SIGUIENTE.

```

p:= PRIMERO(L);
while p <> FIN(L) do begin
  q := p;
  while q <> FIN(L) do begin
    q := SIGUIENTE(q, L);
    r := PRIMERO(L);
    while r <> q do
      r:= SIGUIENTE(r, L)
    end;
    p:= SIGUIENTE(p, L)
  end;
end;

```

- 2.12 Reescribase el código de las operaciones con listas, suponiendo una representación de listas enlazadas, pero sin celdas de encabezado. Supóngase que se usan apuntadores verdaderos y que la posición 1 se representa por nil.
- 2.13 Agréguese al procedimiento de la figura 2.12 todas las verificaciones necesarias para detectar errores.
- 2.14 Otra representación de listas por medio de arreglos consiste en insertar, como en la sección 2.2, pero suprimir simplemente reemplazando el elemento en cuestión por un valor especial «suprimido», que se supone ya no aparece en las listas. Reescribanse las operaciones con listas para aplicar esta estrategia. ¿Cuáles son las ventajas y las desventajas de este enfoque en comparación con la representación original de listas mediante arreglos?

- 2.15 Supóngase que se desea utilizar un bit extra en los registros de colas, que indique si una cola está vacía o no. Modifíquense las declaraciones y las operaciones para una cola circular, de manera que se tenga en cuenta esta característica. ¿Cabe esperar que el cambio valga la pena?
- 2.16 Una *cola doble* o de doble extremo es una lista en la cual se pueden insertar o suprimir elementos en cualquiera de los extremos. Desarrollense realizaciones para colas dobles basadas en arreglos, apuntadores y cursores.
- 2.17 Defínase un TDA que maneje las operaciones PONE_EN_COLA, QUITA_DE_COLA y ESTA_EN_COLA. $ESTA_EN_COLA(x)$ es una función que devuelve verdadero o falso dependiendo de si x está o no en la cola.
- 2.18 ¿Cómo podría obtenerse una cola cuyos elementos son cadenas de longitud arbitraria? ¿Cuánto tiempo lleva poner en cola una cadena?
- 2.19 Otra posible implantación de colas mediante listas enlazadas consiste en no usar una celda de encabezamiento y hacer que *ant* apunte directamente a la primera celda. Si la cola está vacía, se hace que $ant = post = \text{nil}$. Obténgase las operaciones con colas para esta representación. ¿Cómo se compara esta aplicación con la que se dio en la sección 2.4, en función de la velocidad, la utilización de espacio y la concisión del código?
- 2.20 Una variante de la cola circular registra la posición del elemento frontal y la longitud de la cola.
- ¿Es necesario en esta realización limitar la longitud de la cola a $long_máx - 1$?
 - Escríbanse las cinco operaciones con colas para esta realización.
 - Compárese esta realización con la de colas circulares de la sección 2.4.
- 2.21 Es posible guardar dos pilas en un solo arreglo, si una de ellas crece desde la posición 1 del arreglo y la otra lo hace desde la última posición. Escríbase un procedimiento $METE(x, P)$ que inserte el elemento x en la pila P , donde P es una u otra de las dos pilas. Inclúyanse en el procedimiento todas las verificaciones de error necesarias.
- 2.22 Se pueden almacenar k pilas en un solo arreglo si se usa la estructura de datos sugerida en la figura 2.27 para el caso $k = 3$. Para meter y sacar de cada pila se opera como se sugiere en la sección 2.3, en relación con la figura 2.17. Sin embargo, si una inserción en la pila i hace que $TOPE(i)$ iguale a $BASE(i - 1)$, antes de efectuarla hay que desplazar todas las pilas, de modo que quede una separación del tamaño apropiado entre cada par de pilas adyacentes. Por ejemplo, es posible hacer que las separaciones entre pilas sean todas iguales, o que la separación que queda encima de la pila i sea proporcional al tamaño actual de la pila i (en el supuesto de que las pilas más grandes tienen mayor probabilidad de crecer antes, y se desea postergar la siguiente reorganización el mayor tiempo posible).
- Si se dispone de un procedimiento *reorganiza*, al que se llama si las pilas colisionan, escríbase el código de las cinco operaciones con pilas.

- b) Si existe un procedimiento *haz_nuevos_topes* que calcula *nuevo_tope[i]*, la posición «apropiada» para el tope de la pila *i*, con $1 \leq i \leq k$, escríbase el procedimiento *reorganiza*. Sugerencia: Obsérvese que la pila *i* puede moverse arriba o abajo y que hay que mover la pila *i* antes que la *j* si la nueva posición de *j* se traslapa con la posición antigua de *i*. Considérense las pilas 1, 2, 3 ..., *k* en orden, pero manténgase una pila de «objetivos» consistentes en desplazar una pila. Si al considerar la pila *i* ésta se puede mover con seguridad, hágase, y reconsidérese luego la pila cuyo número esté en el tope de la pila de objetivos. Si no hay seguridad, métase *i* en la pila de objetivos.
- c) ¿Qué realización es apropiada para la pila de objetivos de b)? ¿Es realmente necesario mantenerla como una lista de enteros, o sería suficiente con una representación más sucinta?
- d) Obténgase *haz_nuevos_topes* de forma que el espacio que está sobre todas las pilas sea proporcional a los tamaños actuales de éstas.
- e) ¿Qué modificaciones habría que hacer a la figura 2.27 para que esa realización pueda trabajar con colas? ¿Y con listas generales?
- 2.23 Modifíquense las realizaciones de SACA y PONE_EN_COLA de las secciones 2.3 y 2.4, de modo que devuelven el elemento suprimido de la pila o cola. ¿Qué modificaciones habría que hacer si el tipo de los elementos no es alguno que pueda ser devuelto por una función?
- 2.24 Usese una pila para eliminar la recursión de los siguientes procedimientos.

a) **function** *comb* (*n*, *m* : integer): integer;
 {calcula $\binom{n}{m}$ suponiendo que $0 \leq m \leq n$ y $n \geq 1$ }
 begin
 if (*n* = 1) **or** (*m* = 0) **or** (*m* = *n*) **then**
 return (1)
 else
 return (*comb* (*n* - 1, *m*) + *comb*(*n* - 1, *m* - 1))
 end; {*comb*}

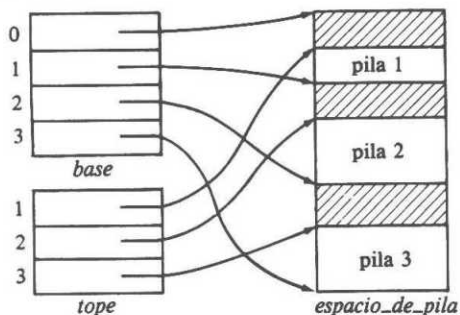


Fig. 2.27. Varias pilas en un arreglo.


```

b) procedure invierte (var L: LISTA);
    { invierte la lista L }
    var
        x: tipo_elemento;
    begin
        if not VACIA(L) then begin
            x := RECUPERA (PRIMERO(L), L);
            SUPRIME(PRIMERO(L), L);
            invierte(L);
            INSERTA(x, FIN(L), L)
        end
    end; { invierte }

```

- *2.25** ¿Es posible eliminar la recursión de cola de los programas del ejercicio 2.24? De ser así, hágase.

Notas bibliográficas

Knuth [1968] contiene información adicional sobre la realización de listas, pilas y colas. Ciertos lenguajes de programación como LISP y SNOBOL, manejan listas y cadenas de manera apropiada. Véanse Sammet [1969], Nicholls [1975], Pratt [1975] o Wexelblat [1981] sobre una historia y una descripción de muchos de estos lenguajes.

3

Arboles

Un árbol impone una estructura jerárquica sobre una colección de objetos. Los árboles genealógicos y los organigramas son ejemplos comunes de árboles. Entre otras cosas, los árboles son útiles para analizar circuitos eléctricos y para representar la estructura de fórmulas matemáticas. También se presentan naturalmente en diversas áreas de computación. Por ejemplo, se usan para organizar información en sistemas de bases de datos y para representar la estructura sintáctica de un programa fuente en los compiladores. En el capítulo 5 se describe el uso de árboles en la representación de datos. En este libro se usan muchas variantes de árboles. En este capítulo se presentan las definiciones básicas y algunas de las operaciones más comunes con árboles. Después se describen algunas de las estructuras de datos más frecuentemente usadas para representar árboles que permiten manejar esas operaciones con eficiencia.

3.1 Terminología fundamental

Un árbol es una colección de elementos llamados *nodos*, uno de los cuales se distingue como *raíz*, junto con una relación (de «paternidad») que impone una estructura jerárquica sobre los nodos. Un nodo, como un elemento de una lista, puede ser del tipo que se desee. A menudo se representa un nodo por medio de una letra, una cadena de caracteres o un círculo con un número en su interior. Formalmente, un *árbol* se puede definir de manera recursiva como sigue:

1. Un solo nodo es, por sí mismo, un árbol. Ese nodo es también la raíz de dicho árbol.
2. Supóngase que n es un nodo y que $A_1, A_2, \dots, A_k$ son árboles con raíces $n_1, n_2, \dots, n_k$, respectivamente. Se puede construir un nuevo árbol haciendo que n se constituya en el padre de los nodos $n_1, n_2, \dots, n_k$. En dicho árbol, n es la raíz y $A_1, A_2, \dots, A_k$ son los *subárboles* de la raíz. Los nodos $n_1, n_2, \dots, n_k$ reciben el nombre de *hijos* del nodo n .

A veces, conviene incluir entre los árboles el *árbol nulo*, un «árbol» sin nodos que se representa mediante Λ .

Ejemplo 3.1. Considérese el índice general de un libro, que se representa en la figura 3.1(a). Tal índice es un árbol. Se puede redibujar en la forma mostrada en la

figura 3.1(b). La relación padre-hijo entre dos nodos se representa por una línea que los une. Los árboles normalmente se dibujan de arriba hacia abajo, como en la figura 3.1(b), con el padre encima de los hijos.

La raíz, el nodo llamado «Libro», tiene tres subárboles que corresponden a los capítulos C1, C2 y C3. Esta relación se representa por medio de las líneas descendentes que unen a Libro con C1, C2 y C3. Libro es el padre de C1, C2 y C3, y estos tres nodos son los hijos de Libro.

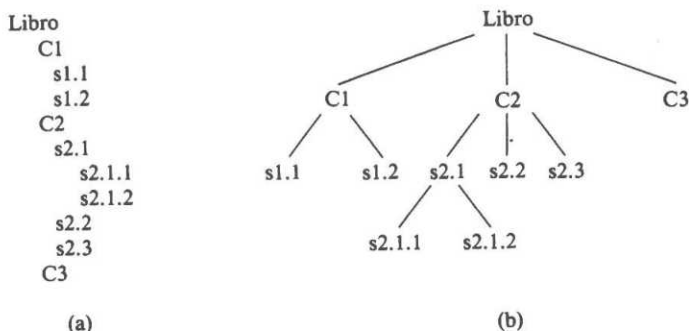


Fig. 3.1. Un índice general y su representación como árbol.

El tercer subárbol de Libro, que tiene raíz C3, es un árbol de un solo nodo, en tanto que los otros dos subárboles tienen una estructura no trivial. Por ejemplo, el subárbol con raíz C2 tiene tres subárboles que corresponden a las secciones s2.1, s2.2 y s2.3; estas dos últimas son árboles de un solo nodo, mientras que la primera tiene dos subárboles que corresponden a las subsecciones s2.1.1 y s2.1.2. □

El ejemplo 3.1 es típico de cierta clase de datos cuya mejor representación se logra mediante un árbol. En el ejemplo, la relación de paternidad representa inclusión: un nodo padre está constituido por sus hijos, como Libro está constituido por C1, C2 y C3. En esta obra se encontrarán otras relaciones que se pueden representar por la relación de paternidad en un árbol.

Si $n_1, n_2, \dots, n_k$, es una sucesión de nodos de un árbol tal que n_i es el padre de n_{i+1} para $1 \leq i < k$, entonces la secuencia se denomina *camino* del nodo n_1 al nodo n_k . La *longitud* de un camino es el número de nodos del camino menos 1. Por tanto, hay un camino de longitud cero de cualquier nodo a sí mismo. Por ejemplo, en la figura 3.1 hay un camino de longitud 2, a saber, (C2, s2.1, s2.1.2), de C2 a s2.1.2.

Si existe un camino de un nodo a a otro b , entonces a es un *antecesor* de b , y b es un *descendiente* de a . Por ejemplo, en la figura 3.1, los antecesores de s2.1 son él mismo, C2 y Libro. Obsérvese que cada nodo es a la vez un antecesor y un descendiente de sí mismo.

Un antecesor o un descendiente de un nodo que no sea él mismo recibe el nombre de *antecesor propio* o *descendiente propio*, respectivamente. En un árbol, la raíz es el único nodo que no tiene antecesores propios. Un nodo sin descendientes pro-

pios se denomina *hoja*. Un subárbol de un árbol es un nodo junto con todos sus descendientes.

La *altura* de un nodo en un árbol es la longitud del camino más largo de ese nodo a una hoja. En la figura 3.1 el nodo C1 tiene altura 1, C2 altura 2 y el nodo C3 altura 0. La *altura del árbol* es la altura de la raíz. La *profundidad* de un nodo es la longitud del camino único desde la raíz a ese nodo.

Orden de los nodos

A menudo los hijos de un nodo se ordenan de izquierda a derecha. Así, los dos árboles de la figura 3.2 son diferentes porque los dos hijos del nodo *a* aparecen en distintos órdenes en los dos árboles. Si se desea ignorar explícitamente el orden de los hijos, se habla entonces de un árbol *no ordenado*.

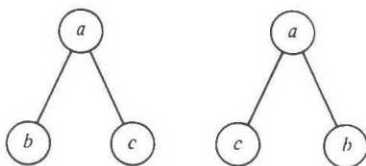


Fig. 3.2. Dos árboles (ordenados) distintos.

El orden de izquierda a derecha de los *hermanos* (hijos del mismo nodo) se puede extender para comparar dos nodos cualesquiera entre los cuales no exista la relación antecesor_descendiente. La regla que se aplica es que si *a* y *b* son hermanos y *a* está a la izquierda de *b*, entonces todos los descendientes de *a* están a la izquierda de todos los descendientes de *b*.

Ejemplo 3.2. Considérese el árbol de la figura 3.3. El nodo 8 está a la derecha del nodo 2, a la izquierda de los nodos 9, 6, 10, 4 y 7, y no está a la izquierda ni a la derecha de sus antecesores 1, 3 y 5.

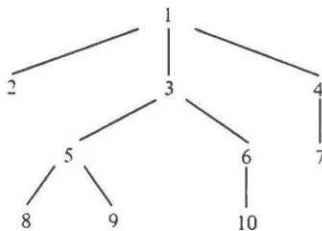


Fig. 3.3. Un árbol.

Dado un nodo n , una regla sencilla para determinar qué nodos están a su izquierda y cuáles a su derecha, consiste en dibujar el camino de la raíz a n . Todos los nodos que salen a la izquierda de este camino, y todos sus descendientes, están a la izquierda de n . Los nodos, y sus descendientes, que salen a la derecha, están a la derecha de n . $\square$

Orden previo, orden posterior y orden simétrico

Existen varias maneras útiles de ordenar sistemáticamente todos los nodos de un árbol. Los tres ordenamientos más importantes se llaman orden previo (*preorder*), orden simétrico (*inorder*) y orden posterior (*postorder*); tales ordenamientos se definen recursivamente como sigue:

- Si un árbol A es nulo, entonces la lista vacía es el listado de los nodos de A en los órdenes previo, simétrico y posterior.
- Si A contiene un solo nodo, entonces ese nodo constituye el listado de los nodos de A en los órdenes previo, simétrico y posterior.

Si ninguno de los anteriores es el caso, sea A un árbol con raíz n y subárboles $A_1, A_2, \dots, A_k$, como se representa en la figura 3.4.

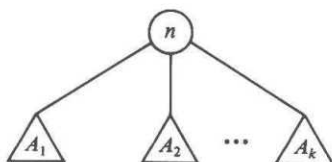


Fig. 3.4. Árbol A .

1. El listado en orden previo (o recorrido en orden previo) de los nodos de A está formado por la raíz de A , seguida de los nodos de A_1 en orden previo, luego por los nodos de A_2 en orden previo y así sucesivamente hasta los nodos de A_k en orden previo.
2. El listado en orden simétrico de los nodos de A está constituido por los nodos de A_1 en orden simétrico, seguidos de n y luego por los nodos de $A_2, \dots, A_k$, con cada grupo de nodos en orden simétrico.
3. El listado en orden posterior de los nodos de A tiene los nodos de A_1 en orden posterior, luego los nodos de A_2 en orden posterior y así sucesivamente hasta los nodos de A_k en orden posterior y por último la raíz n .

La figura 3.5(a) muestra el esbozo de un procedimiento para listar los nodos de un árbol en orden previo. Para convertirlo en un procedimiento de recorrido en orden posterior, basta invertir el orden de los pasos (1) y (2). La figura 3.5(b) es un esbozo de un procedimiento de recorrido en orden simétrico. En los tres casos, el or-

denamiento deseado de un árbol se produce llamando al procedimiento apropiado en la raíz del árbol.

Ejemplo 3.3. Lístense en orden previo los nodos del árbol de la figura 3.3. Primero se lista 1, y luego se llama recursivamente a ORD_PRE en el primer subárbol de 1, o sea el subárbol con raíz 2. Este subárbol tiene un solo nodo, de manera que simplemente se lista. Luego se sigue con el segundo subárbol de 1, el que tiene raíz 3. Se lista 3 y luego se vuelve a llamar a ORD_PRE en el primer subárbol de 3. Esta llamada origina que se listen 5, 8 y 9, en ese orden. Continuando de esta forma, se obtiene el recorrido completo en orden previo de la figura 3.3: 1, 2, 3, 5, 8, 9, 6, 10, 4, 7.

```
procedure ORD_PRE ( n: nodo );
```

```
  begin
```

```
    (1)      lista n;
```

```
    (2)      for cada hijo c de n, si los hay, en orden desde la izquierda do  
              ORD_PRE(c)
```

```
  end; { ORD_PRE }
```

(a) procedimiento ORD_PRE.

```
procedure ORD_SIM ( n: nodo );
```

```
  begin
```

```
    if n es una hoja then
```

```
      lista n
```

```
    else begin
```

```
      ORD_SIM(hijo de n de más a la izquierda);
```

```
      lista n;
```

```
      for cada hijo c de n, excepto el de más a la izquierda, en orden  
        desde la izquierda do
```

```
        ORD_SIM(c)
```

```
    end
```

```
  end; { ORD_SIM }
```

(b) procedimiento ORD_SIM.

Fig. 3.5. Procedimientos recursivos de ordenamiento.

De manera similar, simulando la figura 3.5(a) con el orden de los pasos invertido, se puede descubrir que el recorrido en orden posterior de la figura 3.3 es 2, 8, 9, 5, 10, 6, 3, 7, 4, 1. Simulando la figura 3.5(b), se encuentra que el listado en orden simétrico de la figura 3.3 es 2, 1, 8, 5, 9, 3, 10, 6, 7, 4. □

Un truco útil para producir los tres ordenamientos de nodos es el siguiente. Imagínese que se camina por la periferia del árbol, partiendo de la raíz, y que se avanza en sentido contrario al de las manecillas del reloj, manteniéndose tan cerca del árbol

como sea posible; el camino previsto para la figura 3.3 está representado en la figura 3.6.

Para el orden previo se lista un nodo la primera vez que se pasa por él. En el caso del orden posterior, se lista un nodo la última vez que se pasa por él, conforme se sube hacia su padre. Tratándose del orden simétrico, se lista una hoja la primera vez que se pasa por ella, y un nodo interior, la segunda vez que se pasa por él. A manera de ejemplo, en la figura 3.6 se pasa por el nodo 1 al empezar, y otra vez al pasar por la «bahía» entre los nodos 2 y 3. Obsérvese que el orden de las hojas en los tres ordenamientos corresponde al mismo ordenamiento de izquierda a derecha de las hojas. Sólo el orden de los nodos interiores y su relación con las hojas varía entre los tres ordenamientos.

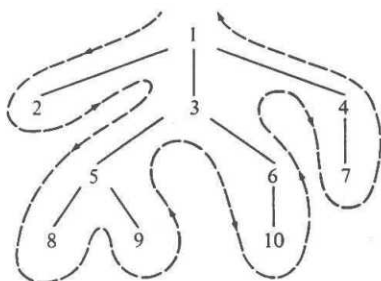


Fig. 3.6. Recorrido de un árbol.

Arboles etiquetados y árboles de expresiones

A menudo es útil asociar una *etiqueta*, o valor, a cada nodo de un árbol, siguiendo la misma idea con que se asoció un valor a un elemento de una lista en el capítulo anterior. Esto es, la etiqueta de un nodo no será el nombre del nodo, sino un valor «almacenado» en él. En algunas aplicaciones, incluso se cambiará la etiqueta de un nodo sin modificar su nombre. Una analogía útil a este respecto es: árbol es a lista como etiqueta es a elemento y nodo es a posición.

Ejemplo 3.4. La figura 3.7 muestra un árbol etiquetado que representa la expresión aritmética $(a + b) * (a + c)$, donde $n_1, n_2, \dots, n_7$ son los nombres de los nodos, cuyas etiquetas se muestran, por convención, en las proximidades de los nodos correspondientes. Las reglas para representar una expresión mediante un árbol etiquetado son estas:

1. Cada hoja está etiquetada con un operando y sólo consta de ese operando. Por ejemplo, el nodo n_4 representa la expresión a .
2. Cada nodo interior n está etiquetado con un operador. Supóngase que n está etiquetado con el operador binario θ , como $+$ o $*$, y que el hijo izquierdo de n re-

presenta la expresión E_1 y el hijo derecho la expresión E_2 . Entonces, n representa la expresión $(E_1) \theta (E_2)$. Los paréntesis se pueden quitar si no son necesarios.

Por ejemplo, el nodo n_2 tiene al operador $+$ como etiqueta, y sus hijos izquierdo y derecho representan las expresiones a y b , respectivamente. Por tanto, n_2 representa $(a) + (b)$, o, más simple, $a + b$. El nodo n_1 representa $(a + b) * (a + c)$, puesto que $*$ es la etiqueta de n_1 , y $a + b$ y $a + c$ son las correspondientes expresiones representadas por n_2 y n_3 . $\square$

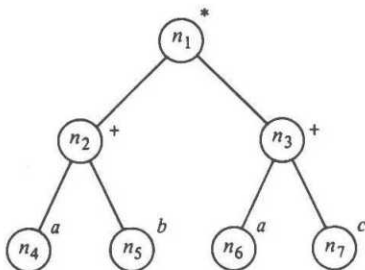


Fig. 3.7. Árbol de expresión con etiquetas.

A menudo, cuando se recorre un árbol en orden previo, simétrico o posterior, se prefiere listar las etiquetas de los nodos, en vez de sus nombres. En el caso de un árbol que representa una expresión, el listado en orden previo de las etiquetas da lo que se conoce como forma *prefija* de la expresión, en la cual un operador precede a sus operandos izquierdo y derecho. Para precisar, la expresión prefija correspondiente a un solo operando a es a mismo. La expresión prefija correspondiente a $(E_1) \theta (E_2)$, donde θ es un operador binario, es $\theta P_1 P_2$, donde P_1 y P_2 son las expresiones prefijas correspondientes a E_1 y E_2 . Obsérvese que en la expresión prefija no se necesitan paréntesis, dado que es posible revisar la expresión prefija $\theta P_1 P_2$ e identificar unívocamente a P_1 como el prefijo más corto (y único) de $P_1 P_2$, que es además una expresión prefija válida.

Por ejemplo, el listado en orden previo de las etiquetas de la figura 3.7 es $++ab+ac$. La expresión prefija correspondiente a n_2 , que es $++ab$, es el prefijo válido más corto de $++ab+ac$.

De manera similar, el listado en orden posterior de las etiquetas de un árbol de expresión da lo que se conoce como representación *postfija* (o *polaca*). La expresión $(E_1) \theta (E_2)$ se representa con la expresión postfija $P_1 P_2 \theta$, donde P_1 y P_2 son las representaciones postfijas de E_1 y E_2 , respectivamente. De nuevo, los paréntesis son innecesarios, porque se puede identificar a P_2 buscando el sufijo más corto de $P_1 P_2$ que sea una expresión postfija válida. Por ejemplo, la expresión postfija correspondiente a la figura 3.7 es $ab+ac+*$. Si se escribe una expresión en la forma $P_1 P_2 *$, entonces P_1 es $ac+$, el sufijo más corto de $ab+ac+$ que es una expresión postfija válida.

El recorrido en orden simétrico de un árbol de expresión da la expresión infija misma, pero sin paréntesis. Por ejemplo, el listado en orden simétrico de las etiquetas de la figura 3.7 es $a+b * a+c$. Se invita a elaborar un algoritmo que recorra un árbol de expresión, y que produzca una expresión infija con todos los pares de paréntesis necesarios.

Determinación de los antecesores

Los recorridos de un árbol en los órdenes previo y posterior permiten determinar los antecesores de un nodo. Supóngase que $ord_post(n)$ es la posición del nodo n en el listado en orden posterior de los nodos de un árbol, y que $desc(n)$ es el número de descendientes propios del nodo n . Por ejemplo, en el árbol de la figura 3.7 se verá cómo las posiciones en orden posterior de los nodos n_2 , n_4 y n_5 son 3, 1 y 2, respectivamente.

Las posiciones en orden posterior de los nodos tienen la útil propiedad de que los nodos de un subárbol con raíz n ocupan posiciones consecutivas de $ord_post(n)-desc(n)$ a $ord_post(n)$. Para determinar si un vértice x es descendiente de un vértice y , basta verificar que se cumple

$$ord_post(y)-desc(y) \leq ord_post(x) \leq ord_post(y)$$

Una propiedad similar se cumple para el recorrido en orden previo.

3.2 EL TDA ARBOL

En el capítulo 2, se vieron las listas, pilas, colas y correspondencias como tipos de datos abstractos (TDA). En este capítulo, los árboles se considerarán como TDA y también como estructuras de datos. Uno de los usos más importantes de los árboles se presenta en el diseño de implantaciones de varios TDA que se estudian en este libro. Por ejemplo, en la sección 5.1 se verá cómo un «árbol binario de búsqueda» puede usarse para obtener tipos de datos abstractos basados en el modelo matemático de un conjunto junto con operaciones como INSERTA, SUPRIME y MIEMBRO (esta última para probar si un elemento es o no elemento de un conjunto). Los dos capítulos siguientes presentan otras realizaciones de varios TDA mediante árboles.

En esta sección se describirán varias operaciones útiles con árboles y se verá cómo diseñar algoritmos para árboles en función de esas operaciones. Igual que con las listas, hay una gran variedad de operaciones que se pueden efectuar con árboles. Aquí se considerarán las siguientes:

1. **PADRE(n, A)**. Esta función devuelve el padre del nodo n en el árbol A . Si n es la raíz, que no tiene padre, se devuelve Λ . En este contexto, Λ es un «nodo nulo», que se usa como señal de que se ha salido del árbol.
2. **HIJO_MAS_IZQ(n, A)** devuelve el hijo más a la izquierda del nodo n en el árbol A , y devuelve Λ si n es una hoja y, por tanto, no tiene hijos.

3. HERMANO_DER(n, A) devuelve el hermano a la derecha del nodo n en el árbol A , el cual se define como el nodo m que tiene el mismo padre p que n , de forma que m está inmediatamente a la derecha de n en el ordenamiento de los hijos de p . Por ejemplo, para el árbol de la figura 3.7, HIJO_MAS_IZQ(n_2) = n_4 , HERMANO_DER(n_4) = n_5 y HERMANO_DER(n_5) = Λ .
4. ETIQUETA(n, A) devuelve la etiqueta del nodo n en el árbol A . Sin embargo, no se requiere que haya etiquetas definidas para cada árbol.
5. CREA i ($v, A_1, A_2, \dots, A_i$) es un miembro de una familia infinita de funciones, una para cada valor de $i = 0, 1, 2, \dots$. CREA i crea un nuevo nodo r con etiqueta v y le asigna i hijos que son las raíces de los árboles $A_1, A_2, \dots, A_i$, en ese orden desde la izquierda. Se devuelve el árbol con raíz r . Obsérvese que si $i = 0$, entonces r es a la vez una hoja y la raíz.
6. RAZ(A) devuelve el nodo raíz del árbol A , o Λ si A es el árbol nulo.
7. ANULA(A) convierte A en el árbol nulo.

Ejemplo 3.5. Escribise un procedimiento recursivo y otro que no lo sea, para listar las etiquetas de los nodos de un árbol en orden previo. Se supone que los tipos de datos nodo y ARBOL ya están definidos, y que el tipo de datos ARBOL es para árboles con etiquetas del tipo tipo-etiqueta. La figura 3.8 muestra un procedimiento recursivo que, dado un nodo n , lista las etiquetas del subárbol con raíz n en orden previo. Para obtener un listado en orden previo correspondiente al árbol A se hace la llamada ORD-PRE(RAZ(A)).

```

procedure ORD_PRE (  $n$ : nodo );
    { lista las etiquetas de los descendientes de  $n$  en orden previo }
    var
         $c$ : nodo;
    begin
        imprime (ETIQUETA( $n, A$ ));
         $c :=$  HIJO_MAS_IZQ( $n, A$ );
        while  $c \neq \Lambda$  do begin
            ORD_PRE( $c$ );
             $c :=$  HERMANO_DER( $c, A$ )
        end
    end; { ORD_PRE }

```

Fig. 3.8. Procedimiento recursivo de listado en orden previo.

También se desarrollará un procedimiento no recursivo para imprimir las etiquetas en orden previo. Para ubicarse en el árbol se usará una pila P , cuyo tipo PILA es, en realidad, «pila de nodos». La idea básica del algoritmo es que cuando se esté en un nodo n , la pila contendrá el camino de la raíz a n , con la raíz en la base de la pila y el nodo n en la parte superior †.

† Recuérdese el análisis sobre recursividad de la sección 2.6, donde se mostró que la implantación de un procedimiento recursivo requiere una pila de registros de activación. Si analiza la figura 3.8, se observa

Una manera de efectuar un recorrido de un árbol en orden previo no recursivo se da en el programa ORD_PRE_NO_REC de la figura 3.9. Este programa tiene dos modos de operación. En el primero, desciende por el camino más a la izquierda aún no explorado del árbol, imprimiendo y apilando los nodos a lo largo del camino, hasta que alcanza una hoja.

Luego, el programa entra en el segundo modo de operación, en el cual regresa por el camino apilado, sacando de la pila los nodos del camino hasta que encuentra un nodo que tiene un hermano derecho. El programa vuelve entonces al primer modo de operación, iniciando el descenso a partir de ese hermano derecho aún no explorado.

El programa empieza en el primer modo de operación en la raíz, y termina cuando la pila queda vacía. En la figura 3.9 se muestra el programa completo.

3.3 Realizaciones de árboles

En esta sección se presentarán varias realizaciones básicas de árboles y se analizarán sus funciones para manejar las operaciones con árboles presentadas en la sección 3.2.

Representación de árboles mediante arreglos

Sea A un árbol cuyos nodos tienen nombres $1, 2, \dots, n$. Tal vez la representación más sencilla de A que es capaz de manejar la operación PADRE sea un arreglo lineal L en la cual cada entrada $L[i]$ constituya un apuntador o un cursor al padre del nodo i . La raíz de A se puede distinguir dándole como padre un apuntador nulo o que apunte a sí misma. En Pascal no se pueden usar apuntadores a elementos de arreglos, por tanto, habrá que emplear un esquema de cursores donde $L[i] = j$ si el nodo j es el padre del nodo i , y $L[i] = 0$ si el nodo i es la raíz.

Esta representación se basa en la propiedad de que en los árboles cada nodo tiene un padre único, y permite hallar el padre de un nodo en un tiempo constante. Un camino ascendente en un árbol, es decir de un nodo a su padre, de éste a su padre y así sucesivamente, se puede recorrer en un tiempo proporcional al número de nodos del camino. También es posible manejar la operación ETIQUETA agregando otro arreglo E , tal que $E[i]$ sea la etiqueta del nodo i , o haciendo que los elementos del arreglo L sean registros que contengan un entero (cursor) y una etiqueta.

Ejemplo 3.6. El árbol de la figura 3.10(a) tiene la representación por apuntadores al padre que se muestra en el arreglo A de la figura 3.10(b). □

que cuando se llama a ORD_PRE (n), las llamadas a procedimientos activas, y por tanto la pila de registros de activación, corresponden a las llamadas a ORD_PRE para todos los antecesores de n . Así, el procedimiento no recursivo de orden previo, como el ejemplo de la sección 2.6, modela con exactitud la forma de implantar el procedimiento recursivo.

La representación por apuntadores al padre no facilita las operaciones que requieren información de los hijos. Dado un nodo n , resulta caro determinar sus hijos o su altura. Además, esta representación no especifica el orden de los hijos de un nodo. Por tanto, las operaciones como HIJO_MAS_IZQ y HERMANO_DER no están bien definidas. Se puede imponer un orden artificial, por ejemplo, numerando los hijos de cada nodo después de numerar al padre, y numerando los hijos en orden

```

procedure ORD_PRE_NO_REC (  $A$ : ARBOL );
    { recorrido en orden previo no recursivo del árbol  $A$  }
    var
         $m$ : nodo; { temporal }
         $P$ : PILA; { pila de nodos que contiene el camino de la raíz al padre TOPE( $P$ ) del nodo "actual"  $m$  }

    begin
        { asigna valor inicial }
        ANULA( $P$ );
         $m$  := RAIZ( $A$ );
        while true do
            if  $m \neq \Lambda$  then begin
                print(ETIQUETA( $m$ ,  $A$ ));
                METE( $m$ ,  $P$ );
                { explora el hijo más a la izquierda de  $m$  }
                 $m$  := HIJO_MAS_IZQ( $m$ ,  $A$ )
            end
            else begin
                { la exploración del camino contenido en la pila está completa }
                if VACIA( $P$ ) then
                    return;
                { explora el hermano derecho del nodo al tope de la pila }
                 $m$  := HERMANO_DER(TOPE( $P$ ),  $A$ );
                SACA( $P$ )
            end
        end; { ORD_PRE_NO_REC }

```

Fig. 3.9. Procedimiento no recursivo de listado en orden previo.

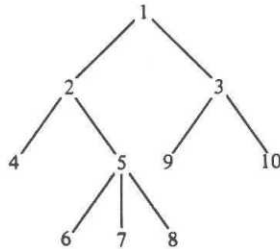
ascendente, de izquierda a derecha. Con esa suposición se ha escrito la función HERMANO_DER de la figura 3.11, para los tipos nodo y ARBOL que se definen como sigue:

```

type
    nodo = integer;
    ARBOL = array [1..nodos_máx] of nodo;

```

Para esta representación se supone que el nodo nulo Λ se representa por 0.



(a) un árbol

	1	2	3	4	5	6	7	8	9	10
A	0	1	1	2	2	5	5	5	3	3

(b) representación por apuntadores al padre.

Fig. 3.10. Un árbol y su representación mediante apuntadores al padre.

```

function HERMANO_DER ( n: nodo; A: ARBOL ) : nodo;
{ devuelve el hermano derecho del nodo n del árbol A }
var
    i, padre: nodo;
begin
    padre := A[n];
    for i := n + 1 to nodos_máx do
        { busca un nodo después de n que tenga el mismo padre }
        if A[i] = padre then
            return (i);
        return (0) { devuelve el nodo nulo si no se encontró hermano
                    derecho }
    end; { HERMANO_DER }
    
```

Fig. 3.11. Operación HERMANO_DER usando la representación por arreglos.

Representación de árboles mediante listas de hijos

Una manera importante y útil de representar árboles consiste en formar una lista de los hijos de cada nodo. Las listas se pueden representar con cualquiera de los métodos sugeridos en el capítulo 2, pero como el número de hijos que cada nodo puede tener es variable, la representación por medio de listas enlazadas es a menudo más apropiada.

La figura 3.12 sugiere cómo se podría representar el árbol de la figura 3.10(a). Hay un arreglo de celdas de encabezamiento indizadas por nodos, los cuales se supone

que están numerados del 1 al 10. Cada encabezamiento apunta a una lista enlazada de «elementos» que son nodos del árbol. Los elementos de la lista encabezada por *encabezamiento[i]* son los hijos del nodo *i*; por ejemplo, 9 y 10 son los hijos de 3.

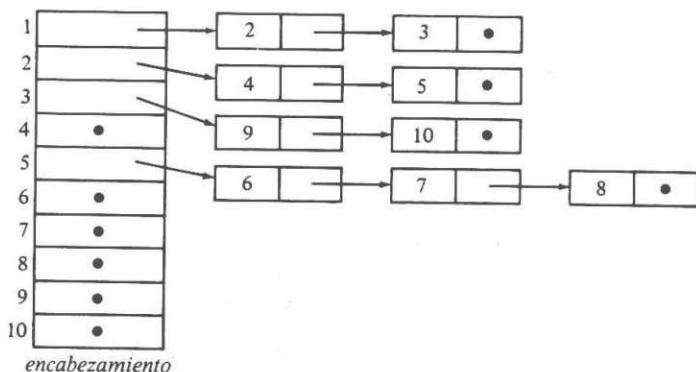


Fig. 3.12. Representación de un árbol por listas enlazadas.

Primero se desarrollarán las estructuras de datos necesarias en función de un tipo de datos abstracto LISTA (de nodos), y luego se dará una realización particular para las listas y se verá cómo las abstracciones compaginan entre sí. Después se mostrarán algunas simplificaciones posibles. Se empieza con las siguientes declaraciones de tipo:

```

type
  nodo = integer;
  LISTA = { definición apropiada para listas de nodos };
  posición = { definición apropiada para posiciones en listas };
  ARBOL = record
    encabezamiento: array [1..nodos_máx] of LISTA;
    etiquetas: array [1..nodos_máx] of tipo_etiqueta;
    raíz: nodo
  end;
```

Se supone que la raíz de cada árbol está almacenada explícitamente en el campo *raíz*, y que se usa 0 para representar el nodo nulo.

La figura 3.13 muestra el código para la operación HIJO_MAS_IZQ. Como ejercicio, sería útil escribir el código para las restantes operaciones.

```

function HIJO_MAS_IZQ ( n: nodo; A: ARBOL ) : nodo;
  { devuelve el hijo más a la izquierda del nodo n del árbol A }
var
  L: LISTA; { abreviatura para la lista de hijos de n }
```

```

begin
  L := A.encabezamiento[n];
  if VACIA(L) then { n es una hoja }
    return (0)
  else
    return (RECUPERA(PRIMERO(L), L))
end; { HIJO_MAS_IZQ }

```

Fig. 3.13. Función para hallar el hijo más a la izquierda.

Ahora se elegirá una implantación particular para las listas, en la cual tanto LISTA como posición sean enteros, empleados como cursores a un arreglo *espacio_celdas* de registros:

```

var
  espacio_celdas: array [1..nodos_máx] of record
    nodo: integer;
    sig: integer
  end;

```

Para simplificar, no se insistirá en que las listas de hijos tienen celdas de encabezamiento. Más bien, se hará que *A.encabezamiento[n]* apunte directamente a la primera celda de la lista, como se sugiere en la figura 3.12. La figura 3.14(a) muestra la función HIJO_MAS_IZQ de la figura 3.13, escrita otra vez para esta realización en particular. La figura 3.14(b) muestra el operador PADRE, que es más difícil de escribir usando esta representación de listas, puesto que se requiere una búsqueda en todas ellas para determinar en cuál aparece un nodo dado.

```

function HIJO_MAS_IZQ ( n: nodo; A: ARBOL ) : nodo;
  { devuelve el hijo más a la izquierda del nodo n del árbol A }
var
  L: integer; { un cursor al principio de la lista de hijos de n }
begin
  L := A.encabezado[n];
  if L = 0 then { n es una hoja }
    return (0)
  else
    return (espacio_celdas[L].nodo)
end; { HIJO_MAS_IZQ }

```

(a) La función HIJO_MAS_IZQ.

```

function PADRE ( n: nodo; A: ARBOL ) : nodo;
  { devuelve el padre del nodo n del árbol A }

```

```

var
  p: nodo; { recorre los posibles padres de n }
  i: posición; { recorre la lista de hijos de p }
begin
  for p := 1 to nodos_máx do begin
    i := A.encabezado[p];
    while i <> 0 do { verifica si n está entre los hijos de p }
      if espacio_celdas[i].nodo = n then
        return (p)
      else
        i := espacio_celdas[i].sig
    end;
  return (0) { devuelve el nodo nulo si no se encontró padre }
end; { PADRE }

```

(b) La función PADRE.

Fig. 3.14. Dos funciones que emplean representación por listas enlazadas de árboles.

Representación «hijo más a la izquierda — hermano derecho»

La estructura de datos descrita antes adolece, entre otras cosas, de no ser adecuada para crear árboles grandes a partir de árboles más chicos por medio de los operadores *CREA_i*. La razón es que, a pesar de que todos los árboles comparten *espacio_celdas* para las listas ligadas de hijos, cada uno tiene su arreglo propio de encabezamientos correspondiente a sus nodos. Por ejemplo, si se deseara obtener *CREA2*(*v*, *A₁*, *A₂*), se tendría que copiar *A₁* y *A₂* en un tercer árbol y agregar un nuevo nodo con etiqueta *v* y dos hijos: las raíces de *A₁* y *A₂*.

Si se desea construir árboles a partir de otros menores, es mejor que las representaciones de los nodos de todos los árboles compartan un área. La extensión lógica de la figura 3.12 para lograr esto consiste en reemplazar el arreglo de encabezamientos por un arreglo *espacio_nodos* que contenga registros con dos campos *etiqueta* y *encabezamiento*. Este arreglo contendrá los encabezamientos de la totalidad de nodos de todos los árboles. Así pues, se declara:

```

var
  espacio_nodos: array [1..nodos_máx] of record
    etiqueta: tipo_etiqueta;
    encabezamiento: integer { cursor a espacio_celdas }
  end;

```

Entonces, puesto que ya no hay nodos con nombres 1, 2, ..., *n*, sino que los nodos están representados por índices arbitrarios de *espacio_nodos*, no es posible que el campo *nodo* de *espacio_celdas* represente el «número» de un nodo; en cambio, *nodo* es ahora un cursor a *espacio_nodos*, que indica la posición de ese nodo. El tipo ARBOL es sencillamente un cursor a *espacio_nodos* que indica la posición de la raíz.

Ejemplo 3.7. La figura 3.15(a) muestra un árbol, y la figura 3.15(b), la estructura de datos correspondiente, en la cual los nodos con etiquetas *A*, *B*, *C*, y *D* se han colocado de manera arbitraria en las posiciones 10, 5, 11 y 2 de *espacio_nodos*. También se ha hecho una elección arbitraria de las celdas de *espacio_celdas* utilizadas para las listas de hijos. □

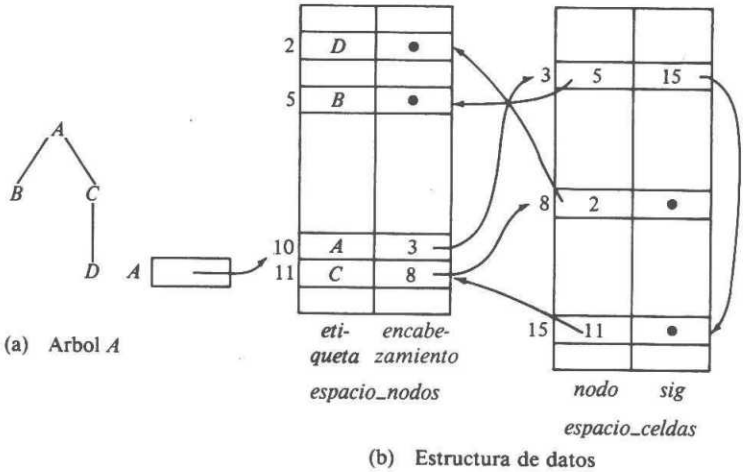


Fig. 3.15. Otra estructura de listas enlazadas para árboles.

La estructura de la figura 3.15(b) es adecuada para combinar árboles por medio de operaciones *CREAi*. Sin embargo, esta estructura se puede simplificar de modo significativo. Primero, se observa que las cadenas de apuntadores *sig* de *espacio_celdas* son en realidad apuntadores a hermanos derechos.

Usando estos apuntadores, se pueden obtener los hijos más a la izquierda como sigue. Supóngase que $\text{espacio_celdas}[i].\text{nodo} = n$. (Recuérdese que el «nombre» de un nodo, en oposición a su etiqueta, es en realidad su índice en *espacio_nodos*, que es el valor de $\text{espacio_celdas}[i].\text{nodo}$.) Entonces, $\text{espacio_nodos}[n].\text{encabezamiento}$ indica la celda ocupada por el hijo más a la izquierda de *n* en *espacio_celdas*, en el sentido de que el campo *nodo* de esa celda es el nombre de tal nodo en *espacio_nodos*.

Se pueden simplificar las cosas si se identifica un nodo no por su índice en *espacio_nodos*, sino por el índice de la celda de *espacio_celdas* que lo representa como hijo. Entonces los apuntadores *sig* de *espacio_celdas* apuntarán en realidad a hermanos derechos, y la información contenida en el arreglo *espacio_nodos* se podrá guardar introduciendo un campo *hijo_más_izq* en *espacio_celdas*. El tipo de datos ARBOL se convertirá en un entero empleado como cursor a *espacio_celdas* para indicar la raíz del árbol. Se declara *espacio_celdas* para tener la siguiente estructura:

```

var
  espacio_celdas: array [1..nodos_máx] of record
    etiqueta: tipo_etiqueta;
    hijo_más_izq: integer;
    hermano_der: integer
  end;

```

Ejemplo 3.8. El árbol de la figura 3.15(a) se representa con la nueva estructura de datos en la figura 3.16. Para los nodos se han utilizado los mismos índices arbitrarios de la figura 3.15(b). □

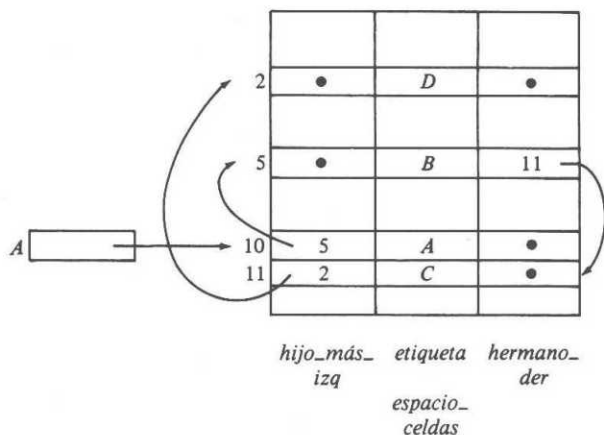


Fig. 3.16. Representación "hijo más a la izquierda-hermano derecho" de un árbol.

En la representación hijo más a la izquierda-hermano derecho, todas las operaciones son fáciles de realizar, excepto PADRE. Esta requiere de una búsqueda en todo *espacio_celdas*. Si es necesario realizar la operación PADRE eficientemente, se puede agregar a *espacio_celdas* un cuarto campo que indique al padre de un nodo directamente.

Como ejemplo de una operación con árboles escrita para utilizar una estructura hijo más a la izquierda-hermano derecho, como la de la figura 3.16, se da la función CREA2 en la figura 3.17. Se supone que las celdas no utilizadas están enlazadas en una lista de espacio disponible, encabezada por *disp*, y que este enlace utiliza los campos hermano derecho. La figura 3.18 muestra los apuntadores antiguos (con líneas de trazo continuo) y los nuevos (con líneas punteadas).

```

function CREA2 ( v: tipo_etiqueta; A1, A2: integer ) : integer;
{ devuelve un nuevo árbol con raíz v, que tiene A1 y A2 como subárboles }
var
    temp: integer; { guarda el índice de la primera celda disponible
                    para la raíz del nuevo árbol }
begin
    temp := disp;
    disp := espacio_celdas[disp].hermano_der,
    espacio_celdas[temp].hijo_más_izq := A1;
    espacio_celdas[temp].etiqueta := v;
    espacio_celdas[temp].hermano_der := 0;
    espacio_celdas[A1].hermano_der := A2;
    espacio_celdas[A2].hermano_der := 0; { no es necesario; ese
        campo debería valer 0, pues la celda era una raíz }
    return (temp)
end; { CREA2 }

```

Fig. 3.17. La función CREA2.

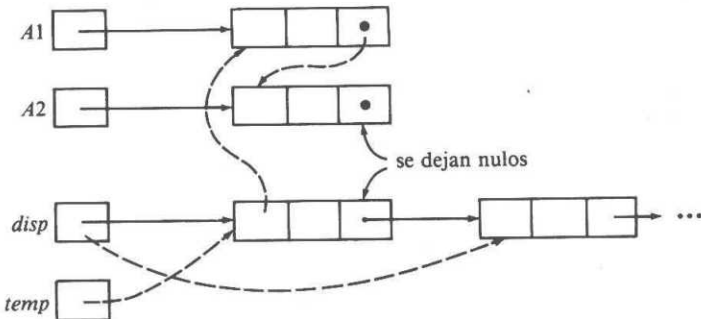


Fig. 3.18. Cambios de apuntadores producidos por CREA2.

Como alternativa, se puede usar menos espacio, pero más tiempo, si se coloca en el campo hermano derecho del hijo más a la derecha un apuntador al padre, en lugar del apuntador nulo que, de lo contrario, estaría ahí. Para evitar confusiones, se necesitaría en cada celda un bit que indicase si el campo hermano derecho tiene un apuntador al hermano derecho o al padre.

Así, dado un nodo, se encontraría su padre siguiendo apuntadores hermano derecho hasta encontrar uno que fuera apuntador al padre. Como todos los hermanos tienen el mismo padre, se encontraría el camino al padre del nodo del que se partió. El tiempo necesario para encontrar el padre de un nodo en esta representación depende del número de hermanos que tenga el nodo.

3.4 Árboles binarios

El árbol que se definió en la sección 3.1 recibe a veces el nombre de árbol *ordenado orientado*, porque los hijos de cada nodo están ordenados de izquierda a derecha, y porque existe un camino orientado (un camino en una dirección particular) de cada nodo a sus descendientes. Otro concepto útil y muy distinto de «árbol» es el de *árbol binario*, que define un árbol vacío o un árbol en el que cada nodo no tiene hijos, tiene un *hijo izquierdo*, un *hijo derecho* o un hijo izquierdo y un hijo derecho. El hecho de que cada hijo se designe en este caso como hijo izquierdo o como hijo derecho hace que un árbol binario sea distinto de un árbol ordenado orientado, tal como se definió en la sección 3.1.

Ejemplo 3.9. Si se adopta la convención de que los hijos izquierdos se dibujan hacia la izquierda de su padre y los hijos derechos hacia la derecha, las figuras 3.19(a) y (b) representan dos árboles binarios distintos, aunque ambos «se parecen» al árbol ordinario (ordenado orientado) de la figura 3.20. Sin embargo, se hace hincapié en que los de la figura 3.19 no son iguales entre sí ni son en ningún sentido iguales al de la figura 3.20, por la sencilla razón de que los árboles binarios no son directamente comparables con los árboles ordinarios. Por ejemplo, en la figura 3.19(a), 2 es el hijo izquierdo de 1, que no tiene hijo derecho; en cambio, en la figura 3.19(b), 1 no tiene hijo izquierdo, pero tiene a 2 como hijo derecho. En ambos árboles binarios, 3 es el hijo izquierdo de 2 y su hijo derecho es 4. □

Los listados en órdenes previo y posterior de un árbol binario son similares a los de un árbol ordinario dados en la página 79. El listado en orden simétrico de los nodos de un árbol binario con raíz n , subárbol izquierdo A_1 y subárbol derecho A_2 , es el listado en orden simétrico de A_1 seguido de n y del listado en orden simétrico de A_2 . Por ejemplo, 35241 es el listado en orden simétrico de los nodos del árbol de la figura 3.19(a).

Representación de árboles binarios

Una forma conveniente de estructurar datos para representar un árbol binario consiste en dar a sus nodos los nombres 1, 2, ..., n , y utilizar un arreglo de registros declarado como

```
var
    espacio_celdas: array [1..nodos_máx] of record
        hijo_izq: integer;
        hijo_der: integer
    end;
```

La idea es que *espacio_celdas[i].hijo_izq* sea el hijo izquierdo del nodo i , y que suceda lo mismo con *hijo_der*. Un valor 0 en cualquiera de esos campos indicará la ausencia de un hijo.

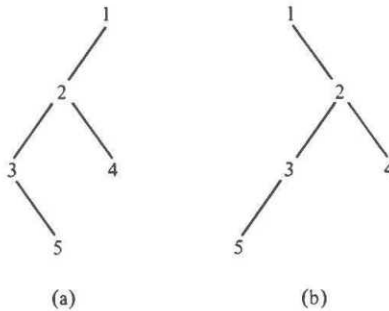


Fig. 3.19. Dos árboles binarios.

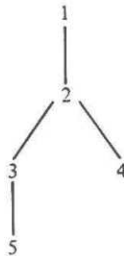


Fig. 3.20. Un árbol «ordinario».

Ejemplo 3.10. El árbol binario de la figura 3.19(a) puede representarse como se muestra en la figura 3.21. □

Un ejemplo: códigos de Huffman

Para ejemplificar la forma en que los árboles binarios pueden usarse como estructuras de datos, se verá un problema particular que consiste en la construcción de «códigos de Huffman». Supóngase que se tienen mensajes compuestos por secuencias de caracteres. En cada mensaje, los caracteres son independientes entre sí, y aparecen con una probabilidad conocida en cualquier posición dada del mensaje; las probabilidades son las mismas para todas las posiciones. Como ejemplo, supóngase que se tiene un mensaje constituido por los caracteres a, b, c, d, e , los cuales aparecen con probabilidades 0.12, 0.4, 0.15, 0.08 y 0.25, respectivamente.

Se desea codificar cada carácter por medio de una sucesión de ceros y unos, de manera que ningún código de un carácter sea prefijo del código de otro carácter. Esta *propiedad de prefijos* permitiría decodificar una cadena de ceros y unos, eliminando repetidas veces de la cadena los prefijos que sean códigos de caracteres.

	<i>hijo_izq</i>	<i>hijo_der</i>
1	2	0
2	3	4
3	0	5
4	0	0
5	0	0

Fig. 3.21. Representación de un árbol binario.

Ejemplo 3.11. La figura 3.22 muestra dos códigos posibles para el alfabeto de cinco símbolos que se está utilizando. Es evidente que el código 1 tiene la propiedad de prefijos, puesto que ninguna sucesión de tres bits puede ser prefijo de otra sucesión con igual número de bits. El algoritmo de decodificación para el código 1 es sencillo. Basta tomar tres bits de cada vez y traducir cada grupo de éstos en un carácter. Por supuesto, las sucesiones 101, 110 y 111 no pueden presentarse si la cadena de bits codifica realmente caracteres de acuerdo con el código 1. Por ejemplo, si se recibe el mensaje codificado 001010011, se puede saber que el mensaje original era *bcd*.

Símbolo	Probabilidad	Código 1	Código 2
<i>a</i>	0.12	000	000
<i>b</i>	0.40	001	11
<i>c</i>	0.15	010	01
<i>d</i>	0.08	011	001
<i>e</i>	0.25	100	10

Fig. 3.22. Dos códigos binarios.

También es fácil verificar que el código 2 tiene la propiedad de prefijos. Se puede decodificar una cadena de bits tomando repetidas veces los prefijos que sean códigos de caracteres y quitándolos, tal como se hizo con el código 1. La única diferencia consiste en que no se puede «rebanar» la sucesión completa de bits de una sola vez, porque tomar dos o tres bits para un carácter depende de los bits. Por ejemplo, si una cadena comienza con 1101001, se puede asegurar de nuevo que los caracteres codificados fueron *bcd*. Los dos primeros bits, 11, deben proceder de *b*, por tanto, es posible quitarlos y preocuparse sólo de 01001. Se deduce, entonces, que los bits 01 proceden de *c*, y así sucesivamente. □

El problema que se enfrenta es: dado un conjunto de caracteres y sus probabilidades, encontrar un código con la propiedad de prefijos, tal que la longitud media del código de un carácter sea mínima. La razón por la que se desea minimizar la longitud media de un código es que ello permite comprimir la longitud de un mensaje tipo. Cuanto más corta sea la longitud media de un carácter, tanto más corta será la longitud de un mensaje codificado. Por ejemplo, el código 1 tiene una longitud

media de código de 3. Esta longitud se obtiene multiplicando las longitudes de los códigos de todos los símbolos por sus respectivas probabilidades de aparición. El código 2 tiene una longitud media de 2.2, puesto que los símbolos a y d , que en conjunto aparecen el 20% de las veces, tienen códigos de longitud 3, y los otros símbolos, códigos de longitud 2.

¿Se puede obtener un código mejor que el 2? Una respuesta completa a esta pregunta consistiría en exhibir un código con la propiedad de prefijos que tuviera una longitud media de 2.15. Ese sería el mejor código posible para las probabilidades dadas de aparición de los símbolos. Una técnica que permite hallar códigos de prefijos óptimos recibe el nombre de *algoritmo de Huffman*. Funciona seleccionando dos caracteres, como a y b , que tengan las probabilidades más pequeñas, y reemplazándolos con un único carácter (imaginario), por ejemplo x , cuya probabilidad de aparición sea la suma de las probabilidades de a y b . Después se encuentra un código de prefijos óptimo para este conjunto más pequeño de caracteres, usando recursivamente el mismo procedimiento. Los códigos para el conjunto original de caracteres se obtienen usando el código para x seguido de un cero, como código para a , y seguido de un uno, como código para b .

Se pueden considerar los códigos de prefijos como caminos en árboles binarios. Piénsese que seguir el camino de un nodo a su hijo izquierdo es como agregarle un cero a un código, y que tomar el camino hacia el hijo derecho es como agregarle un uno. Si se etiquetan las hojas de un árbol binario con los caracteres representados, se puede simbolizar cualquier código de prefijos por medio de un árbol binario. La propiedad de prefijos garantiza que ningún carácter puede tener un código que corresponda a un nodo interior y, recíprocamente, etiquetar las hojas de cualquier árbol binario con caracteres da un código con la propiedad de prefijos de esos caracteres.

Ejemplo 3.12. Los árboles binarios correspondientes a los códigos 1 y 2 de la figura 3.22 se muestran en la figura 3.23(a) y (b), respectivamente. □

Se obtendrá el algoritmo de Huffman mediante un *bosque* o conjunto de árboles, cada uno de los cuales tendrá sus hojas etiquetadas con los caracteres cuyos códigos se desea seleccionar, y sus raíces etiquetadas con la suma de las probabilidades de todas las etiquetas de las hojas. Estas sumas se denominan *pesos* de los árboles. Inicialmente, cada carácter estará en un árbol de un solo nodo, y cuando el algoritmo termine se tendrá un solo árbol con todos los caracteres como hojas. En este árbol, el camino de la raíz a cualquier hoja representará el código para la etiqueta de esa hoja, de acuerdo con el esquema izquierda = 0, derecha = 1 de la figura 3.23.

El paso fundamental del algoritmo consiste en seleccionar los dos árboles del bosque que tengan los pesos más bajos (decidiendo arbitrariamente en caso de empate), y en combinar esos dos árboles en uno solo, cuyo peso sea la suma de los pesos de ambos. Para combinar los árboles se crea un nuevo nodo que queda como raíz que tiene las raíces de los dos árboles dados como sus hijos izquierdo y derecho (sin importar cuál sea uno y otro). Este proceso continúa hasta que quede un solo árbol representativo del código que, para las probabilidades dadas, tenga la menor longitud media de código posible.

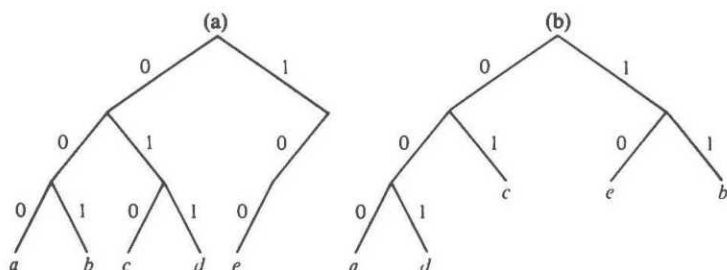


Fig. 3.23. Árboles binarios que representan códigos con la propiedad de prefijos.

Ejemplo 3.13. La secuencia de pasos seguidos para los caracteres y probabilidades del ejemplo que se está desarrollando se muestra en la figura 3.24. En la parte (e), se puede ver que las palabras de código para a , b , c , d y e son 1111, 0, 110, 1110 y 10. En este ejemplo, hay un solo árbol no trivial, pero, en general, puede haber varios. Por ejemplo, si las probabilidades de b y e fueran 0.33 y 0.32, entonces, después del paso (c) de la figura habría que combinar b y e , en vez de añadir e al árbol más grande, como se hizo en el paso (d). □

Se describen ahora las estructuras de datos necesarias. Primero, se usa un arreglo ARBOL de registros del tipo

```
record
    hijo_izq: integer;
    hijo_der: integer;
    padre: integer
end
```

para representar los árboles binarios. Los apuntadores al padre facilitan el hallazgo de caminos de las hojas a la raíz, que permiten descubrir los códigos de los caracteres. En segundo término, se usa un arreglo ALFABETO de registros del tipo

```
record
    simbolo: char;
    probabilidad: real;
    hoja: integer { cursor a un árbol }
end
```

0.12 0.40 0.15 0.08 0.25

• • • • •
a b c d e

(a) Inicial

0.20 0.40 0.15 0.25

•
d a b c e

(b) Combina a y d

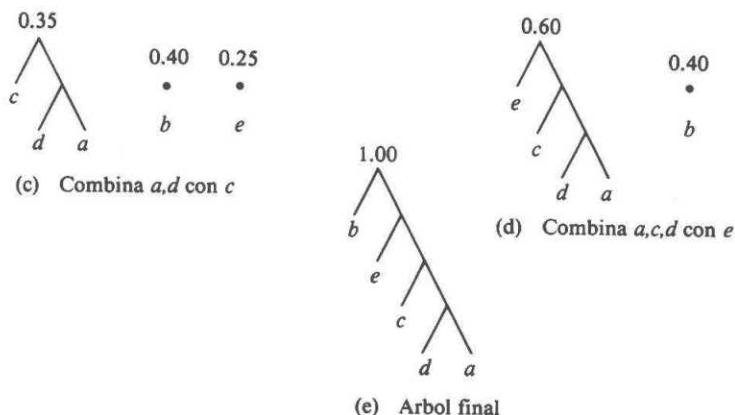


Fig. 3.24. Pasos para la construcción de un árbol de Huffman.

para asociar a cada símbolo del alfabeto a codificar la hoja que le corresponda. Este arreglo también registra la probabilidad de cada carácter. En tercer lugar, se necesita un arreglo *BOSQUE* de registros que representen los árboles mismos. El tipo de esos registros es

```
record
    peso: real;
    raíz: integer { cursor a un árbol }
end
```

Los valores iniciales de todos los arreglos correspondientes a los datos de la figura 3.24(a) se muestran en la figura 3.25. En la figura 3.26 se muestra un esbozo del programa para construir el árbol de Huffman.

1	0.12	1
2	0.40	2
3	0.15	3
4	0.08	4
5	0.25	5

peso raíz

BOSQUE

1	a	0.12	1
2	b	0.40	2
3	c	0.15	3
4	d	0.08	4
5	e	0.25	5

símbolo proba- hoja
lo bilidad

ALFABETO

1	0	0	0
2	0	0	0
3	0	0	0
4	0	0	0
5	0	0	0

hijo_ hijo_ padre
izq der

ARBOL

Fig. 3.25. Contenido inicial de los arreglos.

```

(1)  while haya más de un árbol en el bosque do
      begin
(2)       $i :=$  índice del árbol de menor peso en BOSQUE;
(3)       $j :=$  índice del segundo árbol de menor peso en BOSQUE;
(4)      crea un nuevo nodo con hijo izquierdo BOSQUE[ $i$ ].raíz
          e hijo derecho BOSQUE[ $j$ ].raíz;
(5)      reemplaza el árbol  $i$  de BOSQUE por un árbol cuya
          raíz es el nuevo nodo y cuyo peso es
          BOSQUE[ $i$ ].peso + BOSQUE[ $j$ ].peso;
(6)      elimina el árbol  $j$  de BOSQUE
      end;

```

Fig. 3.26. Esbozo de la construcción de árboles de Huffman.

Para aplicar la línea (4) de la figura 3.26, la cual incrementa el número de celdas empleadas del arreglo *ARBOL*, y las líneas (5) y (6), que disminuyen el número de celdas utilizadas de *BOSQUE*, se introducen los cursores *últ_árbol* y *últ_nodo*, que apuntan hacia *BOSQUE* y *ARBOL*, respectivamente. Se supone que las celdas 1 a *últ_árbol* de *BOSQUE* y 1 a *últ_nodo* de *ARBOL* están ocupadas †. Se supone también que los arreglos de la figura 3.25 tienen declaradas sus longitudes, pero en lo que sigue se omitirán las comparaciones entre esos límites y los valores de los cursores.

```

procedure más_ligeros ( var menor, segundo: integer );
{ asigna menor y segundo a los índices en BOSQUE de los dos árboles
  de menor peso; se supone que últ_árbol  $\geq$  2. }

var
   $i$ : integer;
begin    { asigna valor inicial a menor y a segundo teniendo en cuenta
          los dos primeros árboles }
  if BOSQUE[1].peso  $\leq$  BOSQUE[2].peso then
    begin menor := 1; segundo := 2 end
  else
    begin menor := 2; segundo := 1 end;
  { Ahora recorre  $i$  desde 3 a últ_árbol. En cada iteración, menor es
    el árbol de menor peso entre los  $i$  primeros de BOSQUE; y
    segundo, el siguiente en peso entre los mismos }
  for  $i :=$  3 to últ_árbol do
    if BOSQUE[ $i$ ].peso < BOSQUE[menor].peso then
      begin segundo := menor; menor :=  $i$  end
    else if BOSQUE[ $i$ ].peso < BOSQUE[segundo].peso then
      segundo :=  $i$ 
  end; { más_ligeros }

```

† En la fase de lectura, aquí omitida, también se requiere un cursor para el arreglo *ALFABETO*, mientras se llena con los símbolos y sus probabilidades.

```

function crea ( árbol_izq, árbol_der: integer ) : integer;
{ devuelve un nuevo nodo cuyos hijos son BOSQUE[árbol_izq].
  raíz y BOSQUE[árbol_der].raíz }
begin
  últ_nodo := últ_nodo + 1;
  { la celda para el nuevo nodo es ARBOL[últ_nodo] }
  ARBOL[últ_nodo].hijo_izq := BOSQUE[árbol_izq].raíz;
  ARBOL[últ_nodo].hijo_der := BOSQUE[árbol_der].raíz;
  { ahora asigna apuntadores padre al nuevo nodo y a sus hijos }
  ARBOL[últ_nodo].padre := 0;
  ARBOL[BOSQUE[árbol_izq].raíz].padre := últ_nodo;
  ARBOL[BOSQUE[árbol_der].raíz].padre := últ_nodo;
  return (últ_nodo)
end; { crea }

```

Fig. 3.27. Dos procedimientos.

La figura 3.27 muestra dos procedimientos útiles; el primero es una realización de las líneas (2) y (3) de la figura 3.26, en las que se seleccionan los índices de los dos árboles de menor peso. El segundo es la orden *crea*(n_1, n_2), que crea un nuevo nodo y hace que n_1 y n_2 se conviertan en sus hijos izquierdo y derecho.

Ahora es posible describir con más detalles los pasos de la figura 3.26. En la figura 3.28 se puede ver un procedimiento *Huffman*, que no tiene parámetros sino que trabaja con las estructuras globales de la figura 3.25.

```

procedure Huffman;
var
  i, j: integer; { los dos árboles de menor peso en BOSQUE }
  nueva_raíz: integer;
begin
  while últ_árbol > 1 do begin
    más_ligeros(i, j);
    nueva_raíz := crea(i, j);
    { Ahora reemplaza el árbol i por el árbol cuya raíz es nueva_raíz }
    BOSQUE[i].peso := BOSQUE[i].peso + BOSQUE[j].peso;
    BOSQUE[i].raíz := nueva_raíz;
    { luego reemplaza el árbol j, que ya no se necesita,
      por últ_árbol, y acorta BOSQUE en uno }
    BOSQUE[j] := BOSQUE[últ_árbol];
    últ_árbol := últ_árbol - 1
  end
end; { Huffman }

```

Fig. 3.28. Algoritmo de Huffman.

La figura 3.29 muestra la estructura de datos de la figura 3.25 después de que *últ_árbol* ha quedado reducido a 3; es decir, cuando el bosque tiene el aspecto de la figura 3.24(c).

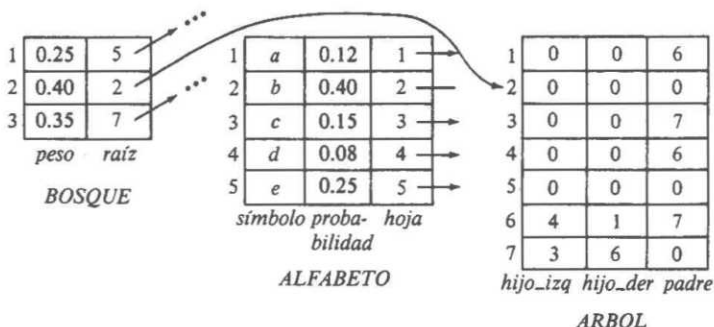


Fig. 3.29. Estructura de datos de un árbol después de dos iteraciones.

Después de terminada la ejecución del algoritmo, el código de cada símbolo se puede determinar como sigue. Encuéntrese el símbolo en el campo *símbolo* del arreglo **ALFABETO**. Sigase el campo *hoja* del mismo registro para obtener un registro del arreglo **ARBOL**; este registro corresponde a la hoja asociada al símbolo en cuestión. Sigase repetidamente el apuntador *padre* desde el registro «actual», por ejemplo, el del nodo *n*, al registro en el arreglo **ARBOL** de su padre *p*. Hágase esto sin olvidar el nodo *n*, de manera que sea posible examinar los apuntadores *hijo_izq* e *hijo_der* de *p* y determinar cuál de ellos apunta a *n*. En el primer caso, imprímase 0; en el segundo, 1. La secuencia de bits impresa de esta forma será el código del símbolo con el orden de los bits invertido. Si se desea imprimir los bits en el orden correcto, se pueden meter en una pila conforme se sube por el árbol, tras lo cual se sacan en forma reiterada bits de la pila, imprimiéndolos al mismo tiempo.

Realización de árboles binarios basada en apuntadores

En vez de utilizar cursores para apuntar a los hijos izquierdos y derechos (y a los padres, si se desea), es posible usar apuntadores genuinos de Pascal. Por ejemplo, se puede declarar

```
type
  nodo = record
    hijo_izq: ↑nodo;
    hijo_der: ↑nodo;
    padre: ↑nodo
  end
```

Por otro lado, si se usara este tipo de nodos para un árbol binario, la función *crea* de la figura 3.27 se escribiría como se muestra en la figura 3.30.

```
function crea ( árbol_izq, árbol_der: ↑ nodo ) : ↑ nodo;
var
    raíz: ↑ nodo;
begin
    new(raíz);
    raíz↑.hijo_izq := árbol_izq;
    raíz↑.hijo_der := árbol_der;
    raíz↑.padre := 0;
    árbol_izq↑.padre := raíz;
    árbol_der↑.padre := raíz;
    return (raíz)
end; { crea }
```

Fig. 3.30. Realización de árboles binarios basada en apuntadores.

Ejercicios

3.1 Respóndase a las siguientes preguntas acerca del árbol de la figura 3.31.

- ¿Qué nodos son hojas?
- ¿Qué nodo es la raíz?
- ¿Cuál es el padre del nodo *C*?
- ¿Qué nodos son los hijos de *E*?
- ¿Qué nodos son los antecesores de *C*?
- ¿Qué nodos son los descendientes de *E*?
- ¿Cuáles son los hermanos derechos de *D* y *E*?
- ¿Qué nodos están a la izquierda y a la derecha de *G*?
- ¿Cuál es la profundidad del nodo *C*?
- ¿Cuál es la altura del nodo *C*?

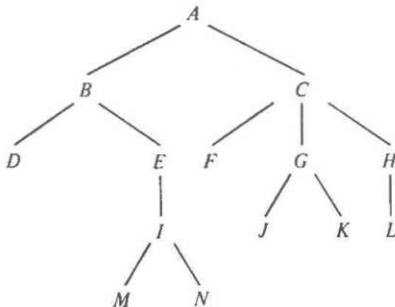


Fig. 3.31. Un árbol.

- 3.2 ¿Cuántos caminos distintos de longitud 3 hay en el árbol representado en la figura 3.31?
- 3.3 Escribanse programas para calcular la altura de un árbol, usando cada una de las tres representaciones de árboles de la sección 3.3.
- 3.4 Lístense los nodos de la figura 3.31 en
- orden previo,
 - orden posterior, y
 - orden simétrico.
- 3.5 Muéstrase que si m y n son dos nodos distintos del mismo árbol, exactamente una de las siguientes afirmaciones es verdadera:
- m está a la izquierda de n
 - m está a la derecha de n
 - m es un antecesor propio de n
 - m es un descendiente propio de n .
- 3.6 Márquese la intersección de la fila i con la columna j si las situaciones representadas por esa fila y esa columna pueden ocurrir simultáneamente.

	$ord_pre(n)$ $< ord_pre(m)$	$ord_sim(n)$ $< ord_sim(m)$	$ord_post(n)$ $< ord_post(m)$
n está a la izquierda de m			
n está a la derecha de m			
n es un antecesor propio de m			
n es un descendiente propio de m			

Por ejemplo, póngase una marca en la intersección de la fila 3 y la columna 2 si se cree que n puede ser a la vez un antecesor propio de m y preceder a m en orden simétrico.

- 3.7 Supóngase que se tienen los arreglos $ORD_PRE[n]$, $ORD_SIM[n]$ y $ORD_POST[n]$ que dan las posiciones en los órdenes previo, simétrico y posterior, respectivamente, de cada nodo n de un árbol. Descríbase un algoritmo que diga si un nodo i es un antecesor o no de un nodo j , para cualquier par de nodos i, j . Explíquese cómo trabaja el algoritmo.
- *3.8 Es posible probar si un nodo m es un descendiente propio o no de un nodo n , determinando si m precede a n en orden X , pero sucede a n en orden Y , donde X e Y se eligen $\{pre, post, sim\}$. Determinense todos los pares X, Y para los cuales se cumple esta afirmación.

- 3.9** Escribbase un programa para recorrer un árbol binario en
- orden previo,
 - orden posterior,
 - orden simétrico.
- 3.10** El listado en *orden de nivel* de los nodos de un árbol lista primero la raíz, luego todos los nodos de profundidad 1, después todos los de profundidad 2, y así sucesivamente. Los nodos que estén a la misma profundidad se listan en orden de izquierda a derecha. Escribbase un programa para listar los nodos de un árbol en orden de nivel.
- 3.11** Conviértase la expresión $((a + b) + c * (d + e) + f) * (g + h)$ en
- expresión prefija
 - expresión postfija.
- 3.12** Dibújense representaciones de árbol para las expresiones
- $a * b * c + de$
 - $a * a + b + cde$.
- 3.13** Sea A un árbol en el que cada nodo que no es hoja tiene dos hijos. Escribbase un programa para convertir
- un listado en orden previo de A en un listado en orden posterior;
 - un listado en orden posterior de A en un listado en orden previo;
 - un listado en orden previo de A en un listado en orden simétrico.
- 3.14** Escribbase un programa para evaluar expresiones aritméticas en
- orden previo
 - orden posterior.
- 3.15** Como modelo matemático, es posible definir un árbol binario como un TDA con estructura de árbol binario y con operaciones como HIJO_IZQ(n), HIJO_DER(n), PADRE(n) y NULO(n). Las tres primeras operaciones devuelven el hijo izquierdo, el hijo derecho y el padre del nodo n (Λ , si no existe), y la última devuelve verdadero si, y sólo si, n es Λ . Implántense estos procedimientos con la representación de árboles binarios de la figura 3.21.
- 3.16** Obténganse las siete operaciones con árboles de la sección 3.2, usando las siguientes realizaciones de árboles:
- apuntadores al padre,
 - listas de hijos,
 - apuntadores al hijo más a la izquierda, hermano derecho.
- 3.17** El *grado* de un nodo es el número de hijos que tiene. Muéstrase que en cualquier árbol binario el número de hojas es uno más que el número de nodos de grado dos.

- 3.18 Muéstrase que el máximo número de nodos de un árbol binario de altura h es $2^{h+1} - 1$. Un árbol binario de altura h con ese máximo número de nodos se denomina árbol binario *lleno*.
- *3.19 Supóngase que los árboles están realizados por medio de apuntadores al hijo más a la izquierda, al hermano derecho y al padre. Elabórense algoritmos no recursivos de recorrido en los órdenes previo, posterior y simétrico, que no utilicen «estados» ni una pila, como se hace en la figura 3.9.
- 3.20 Supóngase que los caracteres a, b, c, d, e , y f tienen probabilidades 0.07, 0.09, 0.12, 0.22, 0.23 y 0.27, respectivamente. Encuéntrese un código de Huffman óptimo y dibújese el árbol de Huffman correspondiente. ¿Cuál es la longitud media del código?
- *3.21 Supóngase que A es un árbol de Huffman y que la hoja del símbolo a tiene una profundidad mayor que la del símbolo b . Demuéstrese que la probabilidad del símbolo b no es menor que la del símbolo a .
- *3.22 Demuéstrese que el algoritmo de Huffman funciona, es decir, que produce un código óptimo para las probabilidades dadas. *Sugerencia*. Utilícese el ejercicio 3.21.

Notas bibliográficas

Berge [1958] y Harary [1969] analizan las propiedades matemáticas de los árboles. Knuth [1973] y Nievergelt [1974] contienen información adicional sobre los árboles de búsqueda binarios. Muchos de los trabajos sobre grafos y sus aplicaciones, a las que se hace referencia en el capítulo 6, también contienen material sobre árboles.

El algoritmo dado en la sección 3.4 para encontrar un árbol con mínima longitud ponderada de camino es de Huffman [1952]. Parker [1980] contiene algunas exploraciones más recientes en ese algoritmo.

4

Operaciones básicas
con conjuntos

Los conjuntos constituyen la estructura básica que fundamenta las matemáticas. En el diseño de algoritmos, los conjuntos se utilizan como la base de una gran cantidad de tipos de datos abstractos y se han desarrollado muchas técnicas para la implantación de tipos de datos abstractos basadas en conjuntos. En este capítulo se hace una revisión de las operaciones básicas con conjuntos y se presentan algunas de las formas más sencillas de realización de conjuntos; se estudian los «diccionarios» y las «colas de prioridad», dos tipos de datos abstractos basados en el modelo de conjunto. Las realizaciones de estos tipos de datos abstractos se cubren en este capítulo y el siguiente.

4.1 Introducción a los conjuntos

Un conjunto es una colección de *miembros* (o *elementos*); cada miembro de un conjunto puede ser un conjunto, o un elemento primitivo que recibe el nombre de *átomo*. Todos los miembros de un conjunto son distintos, lo cual significa que ningún conjunto puede contener dos copias de un mismo elemento.

Por lo general, cuando los átomos se usan como herramientas para el diseño de algoritmos y estructuras de datos, son enteros, caracteres o cadenas, y todos los elementos de cualquier conjunto suelen ser del mismo tipo. A menudo se hará la suposición de que los átomos están ordenados linealmente por una relación que se designa mediante el símbolo « $<$ », y se lee «menor que» o «precede a». Un *orden lineal* $<$ sobre un conjunto S satisface dos propiedades:

1. Para cualesquiera a y b en S , sólo una de las afirmaciones $a < b$, $a = b$ o $b < a$ es verdadera.
2. Para todo a , b y c en S , si $a < b$ y $b < c$, entonces $a < c$ (transitividad).

Los enteros, reales, caracteres y cadenas tienen un orden lineal natural que en Pascal se designa con el símbolo $<$. Se puede definir un orden lineal sobre los objetos que consisten en conjuntos de objetos ordenados. Como ejercicio, determínese una forma de desarrollar un orden de este tipo. Por ejemplo, una pregunta que se debe contestar al desarrollar un orden lineal para un conjunto de enteros es si el conjunto que contiene los enteros 1 y 4 debe considerarse menor o mayor que el conjunto que contiene los enteros 2 y 3.

Notación de conjuntos

Un conjunto de átomos por lo general se representa encerrando sus miembros entre llaves; así, $\{1, 4\}$ denota el conjunto cuyos únicos miembros son 1 y 4. Debe recordarse que un conjunto no es una lista, a pesar de que se representen los conjuntos como si fueran listas; en el caso de los conjuntos, el orden en que se listan los elementos no es importante, y se pudo haber escrito $\{4, 1\}$ en lugar de $\{1, 4\}$. Se debe observar también que cada elemento aparece en un conjunto sólo una vez, de modo que $\{1, 4, 1\}$ no es un conjunto $\dagger$.

Algunas veces se representan los conjuntos mediante una expresión de la forma

$$\{x \mid \text{proposición sobre } x\}$$

donde la proposición sobre x es un predicado que dice con exactitud qué se necesita para que un objeto arbitrario x pertenezca al conjunto. Por ejemplo, $\{x \mid x \text{ es un entero positivo y } x \leq 1000\}$ es otra manera de representar el conjunto $\{1, 2, \dots, 1000\}$, y $\{x \mid \text{para algún entero } y, x = y^2\}$ designa el conjunto de los cuadrados perfectos. Obsérvese que el conjunto de los cuadrados perfectos es infinito y no se puede representar listando sus miembros.

La relación fundamental en teoría de conjuntos es la de pertenencia, que se indica por el símbolo $\in$. Esto es, $x \in A$ significa que x pertenece a A o que el elemento x es un miembro del conjunto A , y puede ser un átomo u otro conjunto, pero A no puede ser un átomo. Se utiliza $x \notin A$ para señalar que « x no es un miembro de A ». Existe un conjunto especial que no tiene miembros, y se simboliza con $\emptyset$, que se denomina *conjunto nulo* o *conjunto vacío*. Obsérvese que $\emptyset$ es un conjunto y no un átomo, a pesar de no tener miembros. La diferencia radica en que $x \in \emptyset$ es falso para toda x , mientras que si y es un átomo, entonces $x \in y$ no tiene sentido, porque más que una afirmación falsa, es un error de sintaxis.

Se dice que un conjunto A está *incluido* (o *contenido*) en un conjunto B , y se escribe $A \subseteq B$ o $B \supseteq A$, si todo miembro de A también es miembro de B . También se dice en este caso que A es un *subconjunto* de B o que B es un *supraconjunto* de A . Por ejemplo, $\{1, 2\} \subseteq \{1, 2, 3\}$, pero $\{1, 2, 3\}$ no es un subconjunto de $\{1, 2\}$ porque 3 pertenece al primero de estos conjuntos, pero no al segundo. Todo conjunto está incluido en sí mismo, y el conjunto vacío está incluido en todo conjunto. Dos conjuntos son iguales si cada uno de ellos está incluido en el otro, esto es, si sus miembros son los mismos. Un conjunto A es un *subconjunto propio* o un *supraconjunto propio* de otro B , si $A \neq B$ y $A \subseteq B$ o $A \supseteq B$, respectivamente.

Las operaciones elementales con conjuntos son unión, intersección y diferencia. Si A y B son conjuntos, entonces $A \cup B$, la *unión* de A y B , es el conjunto de los elementos que son miembros de A , de B o de ambos. La *intersección* de A y B , que se escribe $A \cap B$, es el conjunto de los elementos que pertenecen tanto a A como a B , y la *diferencia*, $A - B$, es el conjunto de los elementos de A que no pertenecen a B .

$\dagger$ Algunas veces el término *conjunto múltiple* se aplica a un «conjunto con repeticiones», es decir, el conjunto múltiple $\{1, 4, 1\}$ tiene (dos veces) 1 y (una vez) 4 como miembros. Un conjunto múltiple tampoco es una lista, por lo que el anterior se pudo haber escrito 4, 1, 1 ó 1, 1, 4.

Por ejemplo, si $A = \{a, b, c\}$ y $B = \{b, d\}$, entonces $A \cup B = \{a, b, c, d\}$, $A \cap B = \{b\}$ y $A - B = \{a, c\}$.

Tipos de datos abstractos basados en conjuntos

Se considerarán ahora los TDA que incorporan diversas operaciones con conjuntos. Algunas colecciones de estas operaciones reciben nombres especiales y tienen realizaciones muy eficientes. Algunas de las operaciones con conjuntos más comunes son las siguientes.

- 1.-3. Los procedimientos $\text{UNION}(A, B, C)$, $\text{INTERSECCION}(A, B, C)$ y $\text{DIFERENCIA}(A, B, C)$ toman los argumentos A y B cuyos valores son conjuntos y asignan el resultado, $A \cup B$, $A \cap B$ o $A - B$, respectivamente, a la variable C de conjuntos.
4. Algunas veces se emplea una operación llamada *combinación* o *unión de conjuntos disjuntos*, que no difiere de la unión, pero supone que sus operandos son disjuntos (no tienen miembros en común). El procedimiento $\text{COMBINA}(A, B, C)$ asigna a la variable C de conjuntos el valor $A \cup B$, pero no está definido cuando $A \cap B \neq \emptyset$, es decir, si los conjuntos A y B no son disjuntos.
5. La función $\text{MIEMBRO}(x, A)$ toma el conjunto A y el objeto x , cuyo tipo es el de los elementos de A , y devuelve un valor booleano: verdadero si $x \in A$ y falso si $x \notin A$.
6. El procedimiento $\text{ANULA}(A)$, hace que el conjunto nulo sea el valor de la variable conjunto A .
7. El procedimiento $\text{INSERTA}(x, A)$, donde A es una variable cuyos valores son conjuntos y x es un elemento del tipo de los miembros de A , hace de x un miembro de A . Esto es, el nuevo valor de A es $A \cup \{x\}$. Obsérvese que si x ya es miembro de A , entonces $\text{INSERTA}(x, A)$ no modifica el conjunto A .
8. $\text{SUPRIME}(x, A)$ elimina x de A , es decir, A se reemplaza por $A - \{x\}$. Si x no está originalmente en A , $\text{SUPRIME}(x, A)$ deja sin cambios el conjunto A .
9. $\text{ASIGNA}(A, B)$ hace que el valor de la variable A de conjuntos sea igual al valor de la variable B , también de conjuntos.
10. La función $\text{MIN}(A)$ devuelve el elemento menor del conjunto A . Esta operación sólo es aplicable cuando los miembros del conjunto parámetro están ordenados linealmente. Por ejemplo, $\text{MIN}(\{2, 3, 1\}) = 1$ y $\text{MIN}(\{'a', 'b', 'c'\}) = 'a'$. También se utiliza la función MAX , cuyo significado es obvio.
11. $\text{IGUAL}(A, B)$ es una función cuyo valor es verdadero si, y sólo si, los conjuntos A y B contienen los mismos elementos.
12. La función $\text{ENCUENTRA}(x)$ opera en un ambiente donde hay una colección de conjuntos disjuntos. $\text{ENCUENTRA}(x)$ devuelve el nombre del (único) conjunto del cual x es miembro.

4.2 Un TDA con UNION, INTERSECCION Y DIFERENCIA

Para empezar, se definirá un TDA para el modelo matemático «conjunto» con las tres operaciones básicas de la teoría de conjuntos, unión, intersección y diferencia. Primero se dará un ejemplo en el que un TDA es útil y después se analizarán varias realizaciones sencillas del mismo.

Ejemplo 4.1. Se desea escribir un programa que efectúe una forma sencilla de «análisis de flujo de datos» sobre diagramas de flujo que representen procedimientos. El programa usará variables de un tipo de datos abstracto CONJUNTO, cuyas operaciones son UNION, INTERSECCION, DIFERENCIA, IGUAL, ASIGNA Y ANULA, según se definieron en la sección anterior.

En la figura 4.1 se observa un diagrama de flujo cuyas casillas tienen los nombres $B_1, \dots, B_8$ y contienen *definiciones de datos* (proposiciones de lectura y asignación) numeradas del 1 al 9. Este diagrama de flujo es una aplicación del algoritmo de Euclides para calcular el máximo común divisor de sus entradas p y q , pero los detalles del algoritmo no tienen importancia para el ejemplo.

En general, un *análisis de flujo de datos* se refiere a la parte de un compilador que examina la representación de un programa fuente en forma de un diagrama de flujo, como la figura 4.1, y reúne información acerca de lo que puede ser cierto conforme el control llega a cada casilla del diagrama. A menudo, las casillas reciben el nombre de *bloques* o *bloques básicos* y representan grupos de proposiciones a través de los cuales el flujo de control procede en forma secuencial. La información que se colecta en el análisis de flujo de datos sirve para mejorar el código generado por el compilador. Por ejemplo, si el análisis permitió determinar que cada vez que el control llegó al bloque B_i , la variable x tuvo el valor 27, entonces se podría sustituir x por 27 en el bloque B_i , a menos que se hubiera asignado un nuevo valor a x dentro de ese bloque. Si el acceso a las constantes fuera más rápido que el acceso a las variables, este cambio haría que el código generado por el compilador se ejecutara a mayor velocidad.

En el ejemplo se desea determinar el lugar donde una variable pudo haber recibido, por última vez, un valor nuevo. Esto es, se desea calcular para cada bloque B_i , el conjunto $DEF_ENT[i]$ de definiciones de datos d , de modo que exista un camino de B_1 a B_i en el cual aparezca d , pero que no esté seguida de otra definición de la misma variable que define d . El conjunto $DEF_ENT[i]$ recibe el nombre de *definiciones de alcance* de B_i .

Para ver cómo podría ser útil esa información, considérese la figura 4.1. El primer bloque B_1 es un bloque «ficticio» con tres definiciones de datos que hacen que las variables t , p y q tomen valores «indefinidos». Si, por ejemplo, se descubre que $DEF_ENT[7]$ incluye la definición 3, la cual da a q un valor indefinido, entonces el programa podría tener un error, puesto que aparentemente podría imprimir q sin antes haberle asignado un valor válido. Por fortuna, se descubrirá que es imposible alcanzar el bloque B_7 sin haber hecho una asignación a q , es decir, 3 no pertenece a $DEF_ENT[7]$.

Varias reglas ayudan a calcular los $DEF_ENT[i]$. Primero, se hace un cálculo pre-

vio, para cada bloque i , de los conjuntos $GEN[i]$ y $ELIM[i]$. $GEN[i]$ es el conjunto de las definiciones de datos del bloque i , con la excepción de que si B_i contiene dos o más definiciones de una variable x , entonces sólo la última de ellas pertenece a $GEN[i]$. Así, $GEN[i]$ es el conjunto de las definiciones de B_i que son «generadas» por B_i ; alcanzan el final de B_i sin que sus variables sean redefinidas.

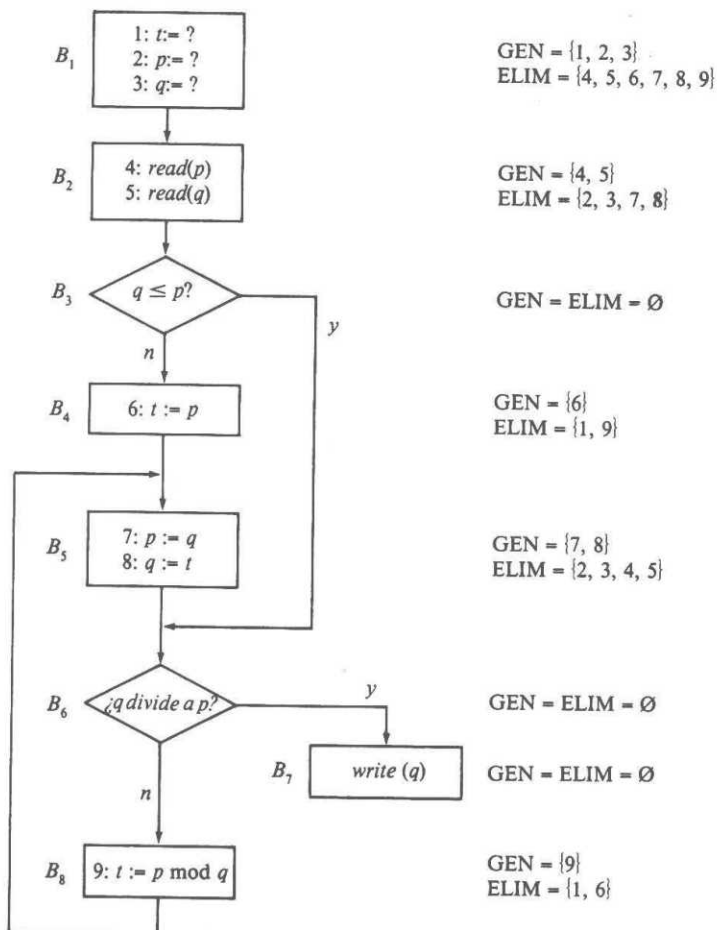


Fig. 4.1. Diagrama de flujo para el algoritmo de Euclides.

El conjunto $ELIM[i]$ es el conjunto de las definiciones d que no están en B_i y tales que B_i contiene una definición de la misma variable que d . Por ejemplo, en la figura 4.1, $GEN[4] = \{6\}$, puesto que la definición 6 (de la variable t) está en B_4 y no hay ninguna definición posterior de t en B_4 . $ELIM[4] = \{1, 9\}$, puesto que 1 y 9 son las definiciones de la variable t que no se encuentran en B_4 .

Además de los $DEF_ENT[i]$, se calcula el conjunto $DEF_SAL[i]$ para cada uno de los bloques B_i . De la misma manera que $DEF_ENT[i]$ es el conjunto de definiciones que alcanzan el principio de B_i , $DEF_SAL[i]$ es el conjunto de definiciones que alcanzan el final de B_i . Existe una fórmula muy sencilla que relaciona DEF_ENT y DEF_SAL :

$$DEF_SAL[i] = (DEF_ENT[i] - ELIM[i]) \cup GEN[i] \quad (4.1)$$

Esto es, la definición d alcanza el final de B_i si, y sólo si, d alcanza el principio de B_i y no es eliminada por B_i o es generada en B_i . La segunda regla que relaciona DEF_ENT y DEF_SAL es que $DEF_ENT[i]$ es la unión, sobre todos los predecesores p de B_i , de $DEF_SAL[p]$, es decir:

$$DEF_ENT[i] = \bigcup_{B_p \text{ precede a } B_i} DEF_SAL[p] \quad (4.2)$$

La regla 4.2 establece que una definición de datos entra en B_i si, y sólo si, alcanza el final de uno de los predecesores de B_i . Como caso especial, si B_i no tiene predecesores, como ocurre con B_1 en la figura 4.1, entonces $DEF_ENT[i] = \emptyset$.

Como ya se han presentado varios conceptos nuevos en este ejemplo, no se profundizará al escribir un algoritmo general para calcular las definiciones de entrada de un diagrama de flujo arbitrario. En vez de ello, se escribirá una parte de un programa que supone que $GEN[i]$ y $ELIM[i]$ se conocen para $i = 1, \dots, 8$, que calcula $DEF_ENT[i]$ y $DEF_SAL[i]$ para $i = 1, \dots, 8$, en el caso particular del diagrama de flujo de la figura 4.1. Este fragmento de programa supone la existencia de un TDA CONJUNTO con las operaciones UNION, INTERSECCION, DIFERENCIA, IGUAL, ASIGNA y ANULA. Más adelante se darán varias implantaciones alternativas de este TDA.

El procedimiento *propaga*($GEN, ELIM, DEF_ENT, DEF_SAL$) aplica la regla 4.1. al cálculo de DEF_SAL para un bloque, dado DEF_ENT . En el caso de un programa sin ciclos, el cálculo de DEF_SAL sería directo. La presencia de un ciclo en este caso exige un procedimiento iterativo. La aproximación a $DEF_ENT[i]$ da comienzo con $DEF_ENT[i] = \emptyset$ y $DEF_SAL[i] = GEN[i]$ para toda i , y después aplica repetidas veces (4.1) y (4.2) hasta que ya no ocurran más cambios en los DEF_ENT y DEF_SAL . Como puede mostrarse que cada nuevo valor asignado a los $DEF_ENT[i]$ y $DEF_SAL[i]$ es un supraconjunto (no necesariamente propio) de su valor anterior, y dado que en cualquier programa hay solamente un número finito de definiciones de datos, el proceso debe converger finalmente a una solución de (4.1) y (4.2).

En la figura 4.3 se muestran los valores sucesivos de $DEF_ENT[i]$ después de cada iteración del ciclo **repeat** de la figura 4.2. Obsérvese que ninguna de las asignaciones ficticias 1, 2 y 3 alcanza un bloque donde se emplea su variable, por tanto, en el programa de la figura 4.1 no hay usos de variables indefinidas. Obsérvese también que al diferir la aplicación de (4.2) para B_i hasta antes de aplicar (4.1) para B_i , haría que en general el proceso de la figura 4.2 convergiera en menos iteraciones. $\square$

4.3 Realización de conjuntos mediante vectores de bits

Obtener la mejor realización para un TDA CONJUNTO depende de las operaciones que se vayan a efectuar y del tamaño de los conjuntos. Cuando todos los conjuntos del dominio en cuestión son subconjuntos de un pequeño «conjunto universal» cuyos elementos son los enteros 1, 2, ..., N , para algún N fijo, se puede usar una realización basada en un *vector de bits* (arreglo booleano). Un conjunto se representa mediante un vector de bits en el que el i -ésimo bit es verdadero si i es un elemento del conjunto. La principal ventaja de esta representación radica en que las operaciones MIEMBRO, INSERTA y SUPRIME se pueden realizar en un tiempo constante mediante una referencia directa al bit apropiado. UNION, INTERSECCION y DIFERENCIA se pueden realizar en un tiempo proporcional al tamaño del conjunto universal.

```

var
  GEN, ELIM, DEF_ENT, DEF_SAL: array[1..8] of CONJUNTO;
  { se supone que GEN y ELIM se calculan por fuera }
  i: integer;
  hay_cambios: boolean;

procedure propaga ( G,K,I: CONJUNTO; var O: CONJUNTO );
  { aplica (4.1) y asigna verdadero a hay_cambios si se detecta algún cambio
    en DEF_SAL }

  var
    TEMP: CONJUNTO;

  begin
    DIFERENCIA(I, K, TEMP);
    UNION(TEMP, G, TEMP);
    if not IGUAL(TEMP, O) do begin
      ASIGNA(O, TEMP);
      hay_cambios := true
    end
  end; { propaga }

begin
  for i := 1 to 8 do
    ASIGNA(DEF_SAL[i], GEN[i]);
  
```

```

repeat
    hay_cambios := false;
    { las ocho proposiciones siguientes aplican (4.2) al diagrama de la figura
      4.1 solamente }
    ANULA(DEF_ENT[1]);
    ASIGNA(DEF_ENT[2], DEF_SAL[1]);
    ASIGNA(DEF_ENT[3], DEF_SAL[2]);
    ASIGNA(DEF_ENT[4], DEF_SAL[3]);
    UNION(DEF_SAL[4], DEF_SAL[8], DEF_ENT[5]);
    UNION(DEF_SAL[3], DEF_SAL[5], DEF_ENT[6]);
    ASIGNA(DEF_ENT[7], DEF_SAL[6]);
    ASIGNA(DEF_ENT[8], DEF_SAL[6]);
    for i := 1 to 8 do
        propaga(GEN[i], ELIM[i], DEF_ENT[i], DEF_SAL[i]);
until
    not hay_cambios
end.

```

Fig. 4.2. Programa para calcular definiciones de alcance.

<i>i</i>	paso 1	paso 2	paso 3	paso 4
1	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
2	{1,2,3}	{1,2,3}	{1,2,3}	{1,2,3}
3	{4,5}	{1,4,5}	{1,4,5}	{1,4,5}
4	$\emptyset$	{4,5}	{1,4,5}	{1,4,5}
5	{6,9}	{6,9}	{4,5,6,7,8,9}	{4,5,6,7,8,9}
6	{7,8}	{4,5,6,7,8,9}	{1,4,5,6,7,8,9}	{1,4,5,6,7,8,9}
7	$\emptyset$	{7,8}	{4,5,6,7,8,9}	{1,4,5,6,7,8,9}
8	$\emptyset$	{7,8}	{4,5,6,7,8,9}	{1,4,5,6,7,8,9}

Fig. 4.3. DEF_ENT [*i*] después de cada iteración.

Si el conjunto universal es suficientemente pequeño para que un vector de bits quepa en una palabra de computador, entonces UNION, INTERSECCION y DIFERENCIA se pueden realizar mediante una sola operación lógica del lenguaje de la máquina en cuestión. Algunos conjuntos pequeños se pueden representar directamente en Pascal por medio de la construcción set. El tamaño máximo de tales conjuntos depende del compilador particular que se esté usando y, por desgracia, a menudo es demasiado pequeño para problemas típicos con conjuntos. Sin embargo, al escribir los propios programas no es necesario restringirse a un límite para el tamaño de los conjuntos, siempre que sea posible tratar cualquier conjunto como un subconjunto de algún conjunto universal $\{1, \dots, N\}$. Se pretende que si A es un conjunto representado por un arreglo booleano, entonces $A[i]$ será verdadero si, y sólo si, el elemento

i es miembro de A . Así, se puede definir un TDA CONJUNTO mediante la declaración en Pascal

```
const
  N = {cualquier valor adecuado};
type
  CONJUNTO = packed array[1..N] of boolean;
```

Se puede implantar entonces el procedimiento UNION tal como se muestra en la figura 4.4. Para implantar INTERSECCION y DIFERENCIA basta reemplazar **or** en la figura 4.4, por **and** y **and not**, respectivamente. Es posible poner en práctica las otras operaciones mencionadas en la sección 4.1 (exceptuando COMBINA y ENCUNETRA, que tienen poco sentido en este contexto) a manera de ejercicios sencillos.

Se puede utilizar la implantación de conjuntos mediante vectores de bits, cuando el conjunto universal es un conjunto finito diferente de un conjunto de enteros consecutivos. De ordinario, se necesitaría una manera de «traducir» entre los miembros del conjunto universal y los enteros 1, ..., N . Así, en el ejemplo 4.1, se supuso que las definiciones de datos tenían asignados números del 1 al 9. En general, las traducciones en ambas direcciones se pueden efectuar mediante el TDA CORRESPONDENCIA descrito en el capítulo 2. Sin embargo, la traducción en sentido inverso de enteros a elementos del conjunto universal puede lograrse mejor con un arreglo A en el cual $A[i]$ sea el elemento correspondiente al entero i .

```
procedure UNION (A, B: CONJUNTO; var C: CONJUNTO);
var
  i: integer;
begin
  for i := 1 to N do
    C[i] := A[i] or B[i];
  end
```

Fig. 4.4. Realización de UNION.

4.4 Realización de conjuntos mediante listas enlazadas

Debe ser evidente que también los conjuntos se pueden representar por medio de listas enlazadas, cuyos elementos son los miembros del conjunto. A diferencia de la representación mediante vectores de bits, la representación mediante listas utiliza un espacio proporcional al tamaño del conjunto representado, y no al del conjunto universal. Más aún, la representación mediante listas es más general, puesto que puede manejar conjuntos que no necesitan ser subconjuntos de algún conjunto universal finito.

Cuando hay operaciones como INTERSECCION con conjuntos representados por listas enlazadas, se presentan varias opciones. Si el conjunto universal tiene un

orden lineal, puede representarse un conjunto por medio de una lista clasificada. Esto es, se supone que todos los miembros de un conjunto son comparables por medio de una relación « $<$ » y los miembros de un conjunto aparecen en una lista en el orden $e_1, e_2, \dots, e_n$, donde $e_1 < e_2 < \dots < e_n$. La ventaja de una lista clasificada radica en que no es necesario revisarla toda para determinar si un elemento se encuentra en la lista.

Un elemento está en la intersección de las listas L_1 y L_2 si, y sólo si, aparece en ambas listas. Para determinar la intersección con listas no clasificadas, es necesario parear cada elemento de L_1 con cada elemento de L_2 en un proceso de $O(n^2)$ pasos con listas de longitud n . La razón por la cual la clasificación de listas facilita la intersección y algunas otras operaciones, consiste en que si se desea parear un elemento e de una lista L_1 con los elementos de otra lista L_2 , sólo es necesario observar los elementos de L_2 hasta que se encuentre e o un elemento mayor que e ; en el primer caso, se habrá encontrado el par, mientras que en el segundo, se sabe que no existe. Es más, si d es el elemento de L_1 que precede inmediatamente a e , y ya se ha encontrado en L_2 el primer elemento, por ejemplo f , tal que $d \leq f$, para buscar en L_2 una aparición de e , se puede empezar con f . La conclusión de este razonamiento es que se pueden encontrar pares para todos los elementos de L_1 , si existen, buscando en L_1 y L_2 una sola vez, siempre que se hagan avanzar los marcadores de posición de las dos listas en el orden apropiado, es decir, avanzando siempre el que apunta al elemento más pequeño. La rutina para realizar INTERSECCION se muestra en la figura 4.5. Ahí, los conjuntos se representan mediante listas enlazadas de «celdas» cuyo tipo se define como

```
type
    tipo_celda = record
        elemento: tipo_elemento;
        sig: ↑ tipo_celda
    end
```

En la figura 4.5, se supone que tipo_elemento es un tipo, por ejemplo un entero, que se puede comparar mediante $<$. En caso contrario, es necesario escribir una función que, dados dos elementos, determine cuál de ellos precede al otro.

Las listas enlazadas de la figura 4.5 están encabezadas por celdas vacías que sirven como puntos de entrada a las listas. Como ejercicio, escríbase este programa en una forma abstracta más general, usando primitivas de listas. Sin embargo, el programa de la figura 4.5 puede ser más eficiente que el programa más abstracto. Por ejemplo, en la figura 4.5 se usan apuntadores a celdas particulares, en vez de variables «de posición» que apunten a celdas anteriores. Se puede hacer esto porque sólo se agrega al final de la lista C , y las listas A y B sólo se revisan sin hacer en ellas inserciones ni supresiones.

Las operaciones UNION y DIFERENCIA se pueden escribir de modo que se asemejen en forma sorprendente al procedimiento INTERSECCION de la figura 4.5. Para UNION, se deben agregar todos los elementos de la lista A o de la B , a la lista C en el orden apropiado de clasificación, de modo que cuando los elementos no sean iguales (líneas 12-14) se agregue el menor a la lista C , como se hace cuando los

```

procedure INTERSECCION ( encab_a, encab_b: ↑ tipo_celda;
    var apc: ↑ tipo_celda );
    { calcula la intersección de las listas clasificadas A y B con celdas de en-
      cabezamiento encab_a y encab_b, dejando el resultado como una
      lista clasificada a cuyo encabezamiento apunta apc }

var
    actual_a, actual_b, actual_c: ↑ tipo_celda;
    { las celdas actuales de las listas A y B y la última celda agregada a la
      lista C }

begin
    (1)   new(apc); { crea el encabezamiento de la lista C }
    (2)   actual_a := encab_a↑.sig;
    (3)   actual_b := encab_b↑.sig;
    (4)   actual_c := apc;
    (5)   while (actual_a <> nil) and (actual_b <> nil) do begin
            { compara los elementos actuales de las listas A y B }
    (6)   if actual_a↑.elemento = actual_b↑.elemento then begin
            { agrega a la intersección }
    (7)   new(actual_c↑.sig);
    (8)   actual_c := actual_c↑.sig;
    (9)   actual_c↑.elemento := actual_a↑.elemento;
    (10)  actual_a := actual_a↑.sig;
    (11)  actual_b := actual_b↑.sig
            end
            else { elementos distintos }
    (12)  if actual_a↑.elemento < actual_b↑.elemento then
    (13)  actual_a := actual_a↑.sig
            else
    (14)  actual_b := actual_b↑.sig
            end;
    (15)  actual_c↑.sig := nil
    end; { INTERSECCION }
    
```

Fig. 4.5. Procedimiento de intersección que utiliza listas clasificadas.

elementos son iguales. También se deben agregar a la lista *C* todos los elementos de la lista que no se haya agotado cuando la prueba de la línea (5) falle. Para DIFERENCIA, no se agrega un elemento a la lista *C* cuando se encuentren elementos iguales; sólo se agrega el elemento actual de la lista *A* a la lista *C* cuando sea menor que el elemento actual de la lista *B*, porque entonces se sabe que el primero no puede encontrarse en la lista *B*. Además, se añaden a *C* los elementos de *A* cuando, y si la prueba de la línea (5) falla porque *B* se agotó.

El operador ASIGNA(*A*, *B*) copia la lista *A* en la lista *B*. Obsérvese que este operador no se puede implantar haciendo simplemente que la celda de encabezamiento de *A* apunte al mismo lugar que la celda de encabezamiento de *B*, porque en tal caso cualquier cambio posterior en *B* causaría cambios no esperados en *A*. El operador MIN

es sencillo; sólo devuelve el primer elemento de la lista. SUPRIME y ENCUENTRA se pueden realizar encontrando el elemento objetivo como se expuso para las listas generales y en el caso de SUPRIME, eliminando su celda.

Por último, la inserción no es difícil de implantar, pero debe hacerse de modo que se inserte el nuevo elemento en la posición apropiada. La figura 4.6 muestra un procedimiento INSERTA que toma como parámetros un elemento y un apuntador a la celda de encabezamiento de una lista, e inserta el elemento en la lista. La figura 4.7 muestra las celdas y apuntadores cruciales antes (líneas de trazo continuo) y después (líneas punteadas) de la inserción.

```

procedure INSERTA ( x: tipo_elemento; a: ↑ tipo_celda );
  { inserta x en la lista a cuyo encabezamiento apunta p }
  var
    actual, cel_nue: ↑ tipo_celda;
  begin
    actual := a;
    while actual↑.sig <> nil do begin
      if actual↑.sig↑.elemento = x then
        return; { si x ya está en la lista, regresa }
      if actual↑.sig↑.elemento > x then
        goto agrega; { salida del ciclo }
      actual := actual↑.sig;
    end;
    agrega: { actual es ahora la celda después de la cual se debe insertar x }
    new(cel_nue);
    cel_nue↑.elemento := x;
    cel_nue↑.sig := actual↑.sig;
    actual↑.sig := cel_nue
  end; { INSERTA }

```

Fig. 4.6. Procedimiento de inserción.

4.5 El diccionario

Cuando se usa un conjunto en el diseño de un algoritmo, quizá no sean necesarias algunas operaciones poderosas como unión e intersección. A menudo, lo único que se necesita es mantener un conjunto de objetos «actuales», con inserciones y supresiones periódicas en el conjunto. Con cierta frecuencia, también puede ser necesario saber si un elemento particular está en el conjunto. Un TDA CONJUNTO con las operaciones INSERTA, SUPRIME y MIEMBRO recibe el nombre de *diccionario*. Se incluirá también ANULA como una operación de diccionario para asignar un valor inicial a cualquier estructura utilizada en la aplicación. Se considerará un ejemplo de realización del diccionario para después analizar algunas realizaciones apropiadas para representar diccionarios.